

データ構造とアルゴリズム

Yuta Ueno

2026 年 7 月 7 日

1 アルゴリズムと計算量

1.1 この章で何を理解するか

この章では、アルゴリズムとは何か、どう書けばよいか、そして複数のアルゴリズムをどう比較するかを学ぶ。講義の中心例は「 n 個の数の最大値を求める問題」であり、そこから計算量、最悪値解析、 O 記号へ進む。以後は、アルゴリズムを「入力から出力を得る厳密な手続き」として読み、その性能を入力サイズの関数として評価する。

この章で固定して読む記号

記号・変数	意味
n	入力の大きさを表す基本変数。問題によってはデータ数を表す。
$x_1, x_2, \dots, x_n$	入力データ。最大値問題では与えられた実数列。
y	計算途中で保持する値。最大値問題では「ここまで見た中の最大値」。
$f(n)$	入力サイズ n のときの実行時間やステップ数を表す関数。
$O(p(n))$	$f(n)$ の成長が高々 $p(n)$ に比例することを表す漸近的表記。

1.2 アルゴリズムとは何か

定義 1.1: アルゴリズム

アルゴリズムとは、与えられたデータから目的の情報を見つけたり、新しい情報を作り出したりするための手続きである。

たとえば、

$$x_1, x_2, \dots, x_n$$

という n 個の実数が与えられたとき、その中の最大値を求める問題を考える。この問題に対しては、「左から順に見ていき、その時点までの最大値を記録しておく」という自然な方針がある。

例 1.1: 最大値問題

25, 59, 5, 66, 6, 83, 7, 41

が与えられたとき、求める最大値は 83 である。

1.3 良いアルゴリズム記述とは何か

講義では、同じ内容でも書き方に良し悪しがあることを強調している。単に「順に見て最大を覚えておく」と言うだけでも意味は伝わるが、入力、出力、手続きが明示されているほうがはるかに良い。

良い記述の条件

- 入力と出力が明確である。
- 何をどの順番で行うかが曖昧でない。
- 必要な変数や代入が明示されている。
- 厳密だが、不要な細部は省いて簡潔である。

この観点から、アルゴリズムは擬似コードで書くことが多い。擬似コードは、実際のプログラム言語ほど細かくはないが、実行手順が十分明確になるように書く記法である。

1.4 最大値を求めるアルゴリズム

最大値問題に対する基本アルゴリズムは次のように書ける。

```
real max_value(int n, real x[1..n]) {  
    real y = x[1];  
    for (int i = 2; i <= n; i++) {  
        if (x[i] > y) {  
            y = x[i];  
        }  
    }  
    return y;  
}
```

このアルゴリズムの考え方

y は常に「これまでに見た要素の最大値」を表すように保つ。新しい要素 x_i を見たとき、

$$x_i > y$$

なら y を更新し、そうでなければ何もしない。最後まで終わった時点で、 y が全体の最大値である。

例 1.2: 不変量の見方

i 番目まで見終わった時点で、

$$y = \max\{x_1, x_2, \dots, x_i\}$$

が成り立っている。この性質が最後の $i = n$ まで保たれるので、返り値は正しい最大値になる。

1.5 正しいが非効率なアルゴリズム

最大値を求めるだけなら、配列全体をソートして最後の値を返す方法も正しい。しかしこれは、最大値だけ欲しいのに、2 番目に大きい値や最小値まで含めて全部の順序を求めていることになる。

```
real bad_max_value(int n, real x[1..n]) {  
    sort x into ascending order;  
    return x[n];  
}
```

注意 1.1: なぜ非効率か

この方法は「必要以上の仕事」をしている。最大値だけなら比較の結果として 1 個だけ分かればよいのに、ソートは全体の並び順を決めようとするからである。

このように、正しさだけでなく効率もアルゴリズム評価の重要な軸になる。

1.6 アルゴリズムをどう評価するか

同じ目的を達成するアルゴリズムは普通いくつもある。そのため、どれが優れているかを判断する基準が必要である。

実際のプログラムを作って実行時間を測る方法は、一見分かりやすいが本質的ではない。なぜなら、実行時間は

- 実装のうまさ
- 計算機の性能
- コンパイラや実行環境

に大きく左右されるからである。

そこで、プログラム化する前の段階で、「基本操作が何回行われるか」を見積もる。これが計算量の考え方である。

1.7 入力サイズ

定義 1.2: 入力サイズ

入力サイズとは、入力データを計算機上で表現するために必要なビット数である。

ただし初学では、問題に応じて「入力データの個数」を入力サイズとみなすことが多い。最大値問題では、まず n 個の数があることが本質なので、入力サイズを n と見て考えると分かりやすい。

注意 1.2: 入力サイズの見方

厳密には、数そのものの大きさを 2 進数で表すためのビット数も入力サイズに影響する。しかし講義の初期段階では、まずデータ数 n による増え方を見るのが基本である。

1.8 時間複雑度の考え方

時間複雑度では、代入、比較、参照、四則演算などの基本操作にかかる時間を定数とみなし、それらが何回行われるかを数える。データによって実行回数が変わるなら、通常は最も時間がかかる場合を考える。

定義 1.3: 時間複雑度

入力サイズ n の関数として表した実行時間の上界を, アルゴリズムの時間複雑度という.

最悪値解析を使う理由

- 入力が違えば同じ n でも実行時間は変わる.
- 最悪値は「この程度以下には必ず収まる」という保証を与える.
- 平均値解析より定義や計算が明確で扱いやすい.

もちろん平均計算量を考えることもあるが, まずは最悪値解析が基本である.

1.9 最大値アルゴリズムの計算量

最大値アルゴリズムを考えると,

- 最初の代入 $y \leftarrow x_1$ は 1 回
- ループは $i = 2, 3, \dots, n$ なので $n - 1$ 回
- 各回で比較を 1 回, 必要なら代入を 1 回

行う.

したがって実行時間は, 定数 α, β を用いて

$$\alpha n + \beta$$

の形で表せる. ここで本当に重要なのは定数の値ではなく, n が大きくなったときの増え方である.

計算量評価の簡略化

実行時間が

$$\alpha n + \beta$$

のように書けると, n が大きくなると支配的なのは 1 次の項である. したがって, このアルゴリズムの計算量は

$$O(n)$$

と評価する.

1.10 漸近的表記 O 記号

定義 1.4: O 記号

自然数上で定義された非負関数 $f(n), p(n)$ に対し,

$$f(n) = O(p(n))$$

とは, ある正定数 C が存在して, すべての十分大きな n に対して

$$f(n) \leq C p(n)$$

が成り立つことをいう.

これは, n を大きくしたとき, $f(n)$ の増え方が高々 $p(n)$ に比例することを意味する. アルゴリズム解析で

は、厳密な値ではなく「どのくらいの速さで増えるか」を見るために使う。

注意 1.3: O 記号の読み方

$f(n) = O(p(n))$ は「 $f(n)$ はだいたい $p(n)$ 以下の速さで増える」と読むとイメージしやすい。ただし等号のように見えても、普通の数式の等式ではなく、成長の上界を表す記号である。

1.11 O 記号の評価例

たとえば

$$f(n) = 3n^2 + 8n + 6$$

について考える。

例 1.3: 2 次式の評価

$n \geq 1$ なら

$$3n^2 + 8n + 6 \leq 3n^2 + 8n^2 + 6n^2 = 17n^2$$

なので、

$$3n^2 + 8n + 6 = O(n^2)$$

である。

同じ関数に対して、

$$O(2n^2), \quad O(n^3)$$

と書くこともできる。なぜなら、それらも n^2 以上に速く増える関数だからである。しかし、できるだけ簡単に鋭い評価を与えるなら $O(n^2)$ が自然である。

例 1.4: なぜ $O(n \log n)$ ではないか

$$\frac{3n^2 + 8n + 6}{n \log n}$$

を考えると、分子の主項は n^2 、分母は $n \log n$ なので、全体はおおよそ

$$\frac{n}{\log n}$$

のように発散する。したがって

$$3n^2 + 8n + 6 \neq O(n \log n)$$

である。

多項式の基本原則

$f(n)$ が k 次多項式なら、最高次項が成長を支配するので

$$f(n) = O(n^k)$$

となる。

1.12 オーダーによるアルゴリズム比較

アルゴリズム A_1, A_2 の計算量がそれぞれ

$$f(n) = O(p(n)), \quad g(n) = O(q(n))$$

だとする。もし

$$p(n) = O(q(n))$$

であり、逆は成り立たないなら、 $p(n)$ のほうが成長が遅い。したがって n が大きくなると、いずれ A_1 のほうが A_2 より速くなる。

例 1.5: 最大値アルゴリズムの比較

最大値を順に比較して求める方法は $O(n)$ である。一方、いったんソートして最後を返す方法は、一般に $O(n \log n)$ である。よって

$$O(n) < O(n \log n)$$

なので、最大値だけを求める目的では前者のほうが優れている。

1.13 代表的なオーダー

計算量にはさまざまな典型形がある。

オーダー	目安
$O(1)$	定数時間。入力サイズによらずほぼ一定。
$O(\log n)$	非常に高速。二分探索などで現れる。
$O(n)$	1 回ずつ全体を見る型。十分高速。
$O(n \log n)$	ソートなどで頻出。実用的で高速。
$O(n^2)$	許容範囲だが、大きい入力では重くなる。
$O(n^3)$	実用上かなり厳しいことが多い。
$O(c^n), O(n!)$	指数オーダー。入力が少し大きいだけで破綻しやすい。

注意 1.4: 多項式時間と指数時間

多項式オーダーのアルゴリズムは一般に「現実的」とみなされる。それに対し、指数オーダーは入力サイズが少し増えただけで実行時間が爆発しやすい。

1.14 時間複雑度と空間複雑度

アルゴリズムの解析では、時間だけでなく記憶領域も重要である。

定義 1.5: 空間複雑度

アルゴリズムの実行に必要な記憶領域を、入力サイズの関数として表したものを空間複雑度という。

したがって、アルゴリズムの複雑度とは

(時間複雑度, 空間複雑度)

の組であると考えられる.

講義では計算機モデルとして, 伝統的な Turing machine と, 実用的な RAM モデルにも触れている. 本章では詳細には立ち入らず, 「基本操作 1 回のコストを定数とみなす」RAM 的な見方で十分である.

1.15 演習を解くためのチェックリスト

- まず入力と出力を明確にし, 何を求めるアルゴリズムかを確認する.
- 途中変数が何を表しているかを言葉で説明できるようにする.
- 計算量は「各基本操作が何回走るか」から見積もる.
- 定数項や低次項は最後に捨て, 支配項だけで O を書く.
- 比較問題では, どちらの成長が遅いかを見る.
- n^k と n^ℓ なら, $k < \ell$ のとき n^k のほうが小さいオーダーである.
- $n, n \log n, n^2, 2^n, n!$ の大小関係はすぐ言えるようにする.

2 リスト構造

2.1 この章で何を理解するか

リスト構造は、データを配列の連続性ではなく「次に進むためのポインタ」でつなぐデータ構造である。講義では名簿管理の例から出発して、なぜ配列だけでは更新が重いのか、その代わりにポインタを持つと何が楽になるのかを説明している。このノートでは、リストの各操作を「どのポインタを書き換えるか」という視点で整理する。

この章で固定して読む記号

記号・変数	意味
L	リスト全体。配列実装では <code>key[]</code> , <code>prev[]</code> , <code>next[]</code> の組として表す。
<code>head</code>	リストの先頭セルのアドレス。空リストなら <code>NIL</code> 。
x	いま注目しているセル。探索の現在地、挿入される新セル、削除対象などで使う。
p	挿入位置の直前のセル、または削除対象 x の直前のセル。
q	挿入・削除時に x の次側にあるセル。
<code>NIL</code>	セルが存在しないことを表す番兵値。配列実装では <code>-1</code> で表すことが多い。
<code>freeL</code>	未使用セルだけをつないだ補助リストの先頭。

2.2 配列管理がづらい理由

名簿のように「順序を保ったまま追加・削除したい」データを、単純な配列で管理すると更新のたびに多くの要素をずらす必要がある。たとえば五十音順の名簿から 1 人削除して別の 1 人を途中に追加すると、後ろにある多数の要素を詰めたり空けたりしなければならない。

配列管理のボトルネック

途中で 1 件削除 $\Rightarrow$ 後続要素を前へ詰める

途中で 1 件追加 $\Rightarrow$ 後続要素を後ろへずらす

したがって、1 回の更新に $O(n)$ の作業がかかる。

この「順番を表すのに、物理的な並び順そのものを使っている」ことが、配列管理を重くする原因である。

2.3 リスト構造の基本アイデア

リストでは、各セルが「次のセルのアドレス」を持つ。順番は配列上の位置ではなく、`next` が作るつながりで決まる。そのため、途中への挿入や途中からの削除では、必要なポインタだけを書き換えればよい。

定義 2.1: 線形リスト

線形リストとは、先頭セルから `next` をたどることで順に全要素へ進めるデータ構造である。最後のセルの `next` は `NIL` である。

定義 2.2: 二重線形リスト

二重線形リストとは, 各セルが前のセルを指す `prev` と, 次のセルを指す `next` の両方を持つリストである. 前後どちらにもたどれるので, 挿入や削除のポインタ更新が書きやすい.

定義 2.3: 環状リスト

環状リストとは, 最後のセルの `next` が先頭へ戻るようにつながったリストである. 末尾に `NIL` がなく, リスト全体が輪になる.

この講義の中心は二重線形リストである. 以後, 各セルは

$$\text{cell} = (\text{prev}, \text{key}, \text{next})$$

として考える.

2.4 リストで行いたい操作

- 探索 (search) : 指定した `key` を持つセルを探す.
- 挿入 (insert) : 新しいセルを指定した位置へ入れる.
- 削除 (delete) : 指定したセルをリストから外す.
- 最小 (minimum) : 最小の `key` を持つセルを返す.
- 最大 (maximum) : 最大の `key` を持つセルを返す.

リストは比較的単純なデータ構造で, これらの操作も「先頭から順に読む」か「前後のポインタをつなぎ直す」かに分解できる.

2.5 配列による実装と動的メモリによる実装

リストの実装方法は大きく 2 つある.

- 構造体と動的メモリ確保を使う方法: `malloc()` や `free()` により必要な分だけセルを用意・解放する.
- 配列で実装する方法: `key[]`, `prev[]`, `next[]` を並べ, 添字をアドレスだと思って扱う.

講義資料では後者の配列実装を使って説明している. このとき「セル」は実体ではなくアドレス番号で識別される.

注意 2.1: 配列実装で見べきもの

配列実装で本当にリストを表しているのは, `head` と `next[]` のつながりである. 添字 0 から順に並んでいること自体には意味がない. 途中の添字が未使用でもまったく問題ない.

2.6 探索: 先頭から順にたどる

List-Search の考え方

探索では, 先頭 head から始めて next を順にたどる. 現在地 x で $\text{key}[x]$ が目標値 a と一致すれば成功, NIL まで行けば失敗である.

```
int list_search(List L, int head, int a) {
    int x = head;
    while (x != NIL && L.key[x] != a) {
        x = L.next[x];
    }
    return x;
}
```

[ここをクリック](#)で映像を確認できます.

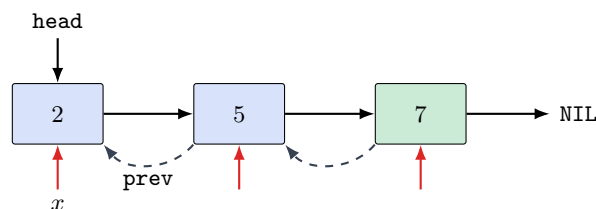


図 1: 探索では head から始めて, 現在地 x を右へ 1 セルずつ進める. 赤い矢印が各反復での x の位置である.

例 2.1: 探索の流れ

head = 2 から始まるリストで key = 9 を探すとする.

2 → 5

の順にセルをたどり, セル 5 で値 9 を見つける. したがって List-Search(L, head, 9) の返り値は 5 である.

最悪の場合はリスト全体を最後まで調べるので, 計算量はリスト長を n として $O(n)$ である.

2.7 挿入: 直前と直後をつなぎ直す

二重線形リストでは, セル p の次へ新しいセル x を挿入する. このとき, q を「もともと p の次にあったセル」とすると,

$$q = \begin{cases} \text{next}[p] & (p \neq \text{NIL}) \\ \text{head} & (p = \text{NIL}) \end{cases}$$

である.

挿入で直す場所

- 新セル側: `prev[x]` と `next[x]` を決める.
- 前側: $p \neq \text{NIL}$ なら `next[p]` を x にする.
- 後側: $q \neq \text{NIL}$ なら `prev[q]` を x にする.
- 先頭挿入なら `head` を x に更新する.

```
int list_insert(List L, int head, int p, int x) {
    int q;
    if (p != NIL) {
        q = L.next[p];
    } else {
        q = head;
    }

    L.prev[x] = p;
    L.next[x] = q;

    if (p != NIL) {
        L.next[p] = x;
    } else {
        head = x;
    }

    if (q != NIL) {
        L.prev[q] = x;
    }

    return head;
}
```

[ここをクリック](#)で映像を確認できます.

例 2.2: 途中への挿入

リスト

$$4 \rightarrow 9 \rightarrow 16$$

で, 値 9 のセルの次に 6 を挿入する. このとき p は値 9 のセル, q は値 16 のセルである. 新セル x を

$$p \rightarrow x \rightarrow q$$

の形に差し込めばよいので, 結果は

$$4 \rightarrow 9 \rightarrow 6 \rightarrow 16$$

となる.

例 2.3: 先頭への挿入

$p = \text{NIL}$ とすると, 先頭挿入になる. このとき古い `head` を q とみなし, `head` を新セル x に更新する.

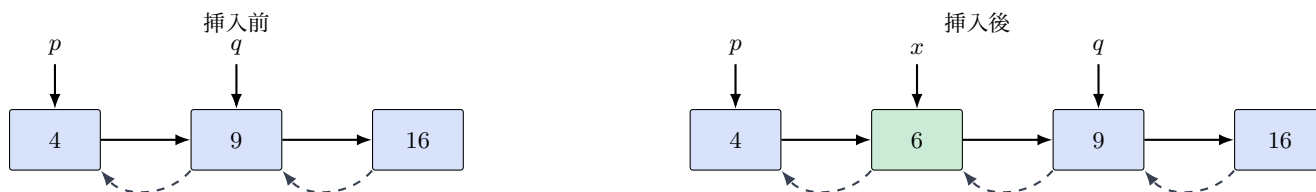


図 2: 途中への挿入では, もとの隣接関係 $p \rightarrow q$ を $p \rightarrow x \rightarrow q$ に置き換える. 更新点は $\text{next}[p]$, $\text{prev}[q]$, $\text{prev}[x]$, $\text{next}[x]$ の 4 箇所である.

ポインタ更新の個数は定数個なので, 挿入の計算量は $O(1)$ である.

2.8 削除: 対象の前後を直接つなぐ

削除では, リスト中に存在するセル x を外す. 二重線形リストなら, 前のセル $p = \text{prev}[x]$ と後ろのセル $q = \text{next}[x]$ がすぐ分かるので, この 2 つを直接つなげばよい.

削除で直す場所

- $p \neq \text{NIL}$ なら $\text{next}[p]$ を q にする.
- $p = \text{NIL}$ なら, 削除対象は先頭なので head を q にする.
- $q \neq \text{NIL}$ なら $\text{prev}[q]$ を p にする.

```
int list_delete(List L, int head, int x) {
    int p = L.prev[x];
    int q = L.next[x];

    if (p != NIL) {
        L.next[p] = q;
    } else {
        head = q;
    }

    if (q != NIL) {
        L.prev[q] = p;
    }

    return head;
}
```

[ここをクリック](#)で映像を確認できます.

例 2.4: 途中のセルの削除

リスト

$4 \rightarrow 9 \rightarrow 16$

から 9 を持つセルを削除する. 前後のセル 4 と 16 を直接つなげばよいので, 結果は

$4 \rightarrow 16$

になる.



図 3: 削除では, 対象 x の前後にある p と q を直接つなぐ. すると x は本体リストの経路から外れる.

注意 2.2: 削除したセルの配列内容はどうするか

配列実装では, 削除後に `key[x]`, `prev[x]`, `next[x]` を空に戻すかどうかは実装次第である. 重要なのは「リスト本体からは切り離された」ことであり, そのセルを再利用するなら別途 `freeL` で管理する.

削除もポインタ更新は定数個なので $O(1)$ である. ただし「`key = a` のセルを削除したい」ときは, 先に探索が必要なので全体では $O(n)$ になる.

2.9 最大値と最小値

リストには二分探索木のような大小関係の埋め込みがない. したがって最大値・最小値を求めるには, 先頭から末尾まで全部見て比較するしかない.

```
int list_maximum(List L, int head) {
    int x = head;
    int y = x;
    int max = -INF;
    while (x != NIL) {
        if (L.key[x] > max) {
            max = L.key[x];
            y = x;
        }
        x = L.next[x];
    }
    return y;
}

int list_minimum(List L, int head) {
    int x = head;
    int y = x;
    int min = INF;
    while (x != NIL) {
        if (L.key[x] < min) {
            min = L.key[x];
            y = x;
        }
        x = L.next[x];
    }
    return y;
}
```

```

    }
    x = L.next[x];
}
return y;
}

```

[ここをクリック](#)で映像を確認できます。

なぜ全探索が必要か

リストは「順番に並んでいる」だけで、大小関係に従って整列されているとは限らない。そのため、途中で見ただけでは残りにもっと大きい値、もっと小さい値があるか分からない。

したがって最大・最小の計算量も $O(n)$ である。

2.10 未使用セルの管理: freeL

配列実装では、挿入のたびに「まだ使っていない添字」を 1 つ取り出す必要がある。また、削除したセルは将来の挿入で再利用したい。このために、未使用セルだけをつないだ補助リスト **freeL** を用いる。

定義 2.4: free list

freeL とは、現在リスト本体では使われていないセルだけを **next** でつないだリストである。先頭は変数 **freeL** が指し、各未使用セルの **next** を用いて管理する。

freeL の運用

- 挿入前: **freeL** の先頭セルを 1 個取り出す。
- 削除後: 外したセルを **freeL** の先頭へ戻す。

これにより、空きセルの取得と返却をどちらも $O(1)$ で行える。

```

int allocate_object(List L, int *freeL) {
    int x;
    if (*freeL == NIL) {
        error("out of space");
    }
    x = *freeL;
    *freeL = L.next[x];
    return x;
}

void free_object(List L, int *freeL, int x) {
    L.next[x] = *freeL;
    *freeL = x;
}

```

[ここをクリック](#)で映像を確認できます。

例 2.5: freeL からの取り出し

freeL = 3 なら, まずセル 3 を未使用リストから取り出し,

$$\text{freeL} \leftarrow \text{next}[3]$$

と更新する. その後 key[3] に値を入れて List-Insert を呼べば, 新しいセルとして利用できる.

例 2.6: 削除セルの返却

List-Delete でセル 1 を本体リストから外したら,

$$\text{next}[1] \leftarrow \text{freeL}, \quad \text{freeL} \leftarrow 1$$

とすれば, セル 1 は再び未使用セルとして保存される.

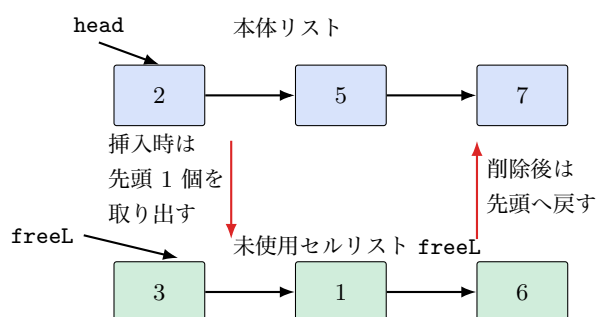


図 4: 配列実装では, 本体リストと未使用セルリスト freeL を別々に管理する. セルの取得も返却も先頭で行うため, どちらも $O(1)$ である.

2.11 計算量のまとめ

操作	計算量
Search	$O(n)$
Insert	$O(1)$
Delete	$O(1)$
Maximum, Minimum	$O(n)$
Allocate-Object, Free-Object	$O(1)$

ここで n はリストの長さである. リストは挿入・削除に強い一方, 値を探したり最大最小を求めたりする操作には線形時間がかかる.

2.12 線形リストと二重線形リストの違い

注意 2.3: なぜ prev があると削除が楽か

線形リストでは prev がないので、セル x を削除するには「 x の 1 つ前のセル」を別途見つける必要がある。そのため、削除だけ見れば二重線形リストのほうが扱いやすい。

逆に、線形リストは各セルが持つ情報が少ないので省メモリである。どちらを選ぶかは、更新のしやすさとメモリ量のトレードオフで決まる。

2.13 演習を解くためのチェックリスト

- 配列が与えられたら、まず head から next をたどって本体リストを図にする。
- prev は「戻る向き」、next は「進む向き」として必ず対で確認する。
- 挿入では、新セル x の前後が誰かを最初に決めてからポインタを書き換える。
- 削除では、prev[x] と next[x] を先に読み取り、その前後を直接つなぐ。
- freeL がある問題では、本体リストと未使用セルリストを混同しない。
- 計算量は「先頭から順に全部見るか」「定数本のポインタ更新だけか」で判定する。

3 二分探索木

3.1 この章で何を理解するか

二分探索木 (binary search tree; BST) は, key の大小関係を木の形に埋め込むデータ構造である. 講義資料では図とコードが別々に出てくるため, まず「ポインタが何を指すか」と「変数が探索中に何を表すか」を固定して読む.

このノートで使う読み替え

記号・変数	意味
T	木全体. 実装上は根を指すポインタを持てば十分.
root	木の根. C 言語コードでは引数 root や戻り値がこれを表す.
x	いま注目している頂点. 探索では現在地, 挿入では新しく入れる頂点.
y	挿入・削除で「親になる頂点」または「実際に取り除く頂点」. 文脈で役割が変わる.
z	挿入では場所探し用, 削除では消したい頂点.
p	親頂点. 削除でポインタを張り替える相手.
NIL	頂点が存在しないことを表す番兵値. C では NULL.

3.2 木と二分木の最小限の用語

定義 3.1: 根付き木

根付き木とは, 特別な頂点である根を 1 つ選んだ木である. 根から各頂点へのパスはちょうど 1 本なので, 親, 子, 祖先, 子孫, 深さ, 高さが定義できる.

- 頂点の深さ: 根からその頂点までの辺の本数.
- 木の高さ: 頂点の深さの最大値.
- 葉: 子を持たない頂点.
- 部分木: ある頂点とその子孫だけからなる木.

定義 3.2: 二分木

二分木とは, 各頂点が高々 2 個の子を持つ根付き木である. 子は左の子 left と右の子 right として区別する.

ポインタ実装では, 各頂点を次の 4 つ組として見る.

$$\text{node} = (\text{key}, \text{parent}, \text{left}, \text{right})$$

この章の操作はすべて, この 4 本の情報をどう読むか, どう張り替えるかで決まる.

3.3 二分探索木の不変条件

定義 3.3: 二分探索木

二分探索木とは, 各頂点に key を持つ二分木で, 任意の頂点 x について次を満たすものをいう.

$$\begin{aligned} y \in x \text{ の左部分木} &\implies \text{key}[y] \leq \text{key}[x], \\ y \in x \text{ の右部分木} &\implies \text{key}[x] \leq \text{key}[y]. \end{aligned}$$

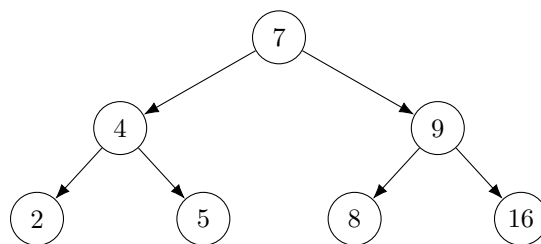


図 5: 二分探索木の例. 左へ行くほど小さく, 右へ行くほど大きい.

注意 3.1: 図を見るとき注意

同じ key 集合でも二分探索木の形は一意ではない. たとえば 2, 4, 5, 7, 8, 9, 16 をどの順番で挿入するかによって, ほぼ完全な木にも, 片側に伸びたリスト状の木にもなる. 操作時間は key の個数だけでなく, 高さ h に強く依存する.

3.4 探索: 進む方向は比較 1 回で決まる

Tree-Search の考え方

探したい値を a とする. 現在地 x で $\text{key}[x]$ と a を比べる.

$$a < \text{key}[x] \Rightarrow \text{左へ進む}, \quad a > \text{key}[x] \Rightarrow \text{右へ進む}.$$

一致したら発見, NIL に到達したら存在しない.

```
Node *tree_search(Node *x, int a) {
    while (x != NULL && x->key != a) {
        if (a < x->key) {
            x = x->left;
        } else {
            x = x->right;
        }
    }
    return x;
}
```

[ここをクリック](#)で映像を確認できます.

例 3.1: 探索の道筋

図の木で $a = 8$ を探す.

$8 > 7 \Rightarrow$ 右へ, $8 < 9 \Rightarrow$ 左へ, $8 = 8 \Rightarrow$ 発見.

見た頂点は $7 \rightarrow 9 \rightarrow 8$ だけであり, 他の頂点は調べない.

根から葉へ 1 段ずつ降りるだけなので, 計算量は木の高さを h として $O(h)$ である.

3.5 最小値と最大値

二分探索木では, 最小値は「左へ進めるだけ進んだ頂点」, 最大値は「右へ進めるだけ進んだ頂点」である.

```
Node *tree_minimum(Node *x) {
    while (x->left != NULL) {
        x = x->left;
    }
    return x;
}

Node *tree_maximum(Node *x) {
    while (x->right != NULL) {
        x = x->right;
    }
    return x;
}
```

[ここをクリック](#)で映像を確認できます.

なぜこれでよいか

左部分木の key は必ず現在の key 以下である. したがって, 左の子がある限り, 現在地はまだ最小ではない. 左の子がなくなった時点で, それ以上小さい方向へ進めないで, その部分木の最小値である. 最大値は左右を入れ替えた同じ議論でよい.

3.6 挿入: 最後は必ず葉として入る

挿入では, 新しい頂点 x の key だけが決まっているとする. 左子と右子はまだ存在しないので,

$\text{left}[x] = \text{right}[x] = \text{NIL}$

としておく.

挿入で使う 2 つの変数

- z : 木を降りていく現在地. 最後は NIL になる.
- y : z の 1 つ前にいた頂点. 最後は x の親になる.

```
Node *tree_insert(Node *root, Node *x) {
    Node *y = NULL;
```

```

Node *z = root;
while (z != NULL) {
    y = z;
    if (x->key < z->key) {
        z = z->left;
    } else {
        z = z->right;
    }
}

x->parent = y;
if (y == NULL) {
    root = x;
} else if (x->key < y->key) {
    y->left = x;
} else {
    y->right = x;
}
return root;
}

```

[ここをクリック](#)で映像を確認できます。

例 3.2: 14 を挿入する

次の木に 14 を挿入する。

$15 \rightarrow 6 \rightarrow 8 \rightarrow 13$

の順に比較し, $14 > 13$ なので, 13 の右の子として入る. このとき最後の有効な頂点 y は 13, 現在地 z は NIL である.

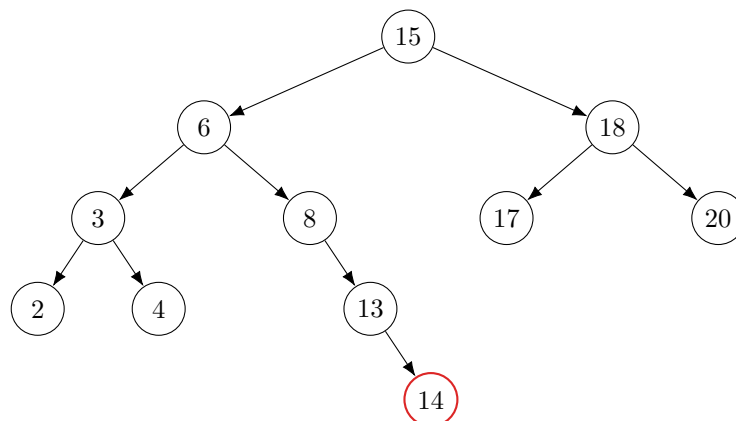


図 6: 挿入後の 14. 新しい頂点は葉として追加される.

3.7 削除: 消す頂点と実際に取り除く頂点を分ける

削除がわかりづらい理由は、「指定された頂点 z 」と「実際にメモリから取り除く頂点 y 」が一致しない場合があるからである。

削除の 3 ケース

z の子の数	方針	実際に消える頂点
0 個	親から z へのポインタを NIL にする	z
1 個	z の場所に唯一の子を上げる	z
2 個	z の次に大きい頂点 y の key を z にコピーし, y を消す	y

子が 0 個または 1 個の場合

この場合, z を取り除いても, 残りの部分木を 1 本つなぎ直すだけでよい. 唯一の子を x , 親を p とすると, 必要な更新は次である.

$$\text{parent}[x] \leftarrow p, \quad \text{left}[p] \text{ または } \text{right}[p] \leftarrow x.$$

$x = \text{NIL}$ なら, 親からのポインタを NIL にするだけである.

子が 2 個の場合

z の左右に子がある場合, z そのものを単純に消すと左右の部分木を同時につなぎ直す必要がある. そこで, z の右部分木の最小値を持つ頂点

$$y = \text{Tree-Minimum}(T, \text{right}[z])$$

を使う. この y は z より大きい key の中で最小なので, z の位置に key をコピーしても二分探索木の条件を壊さない.

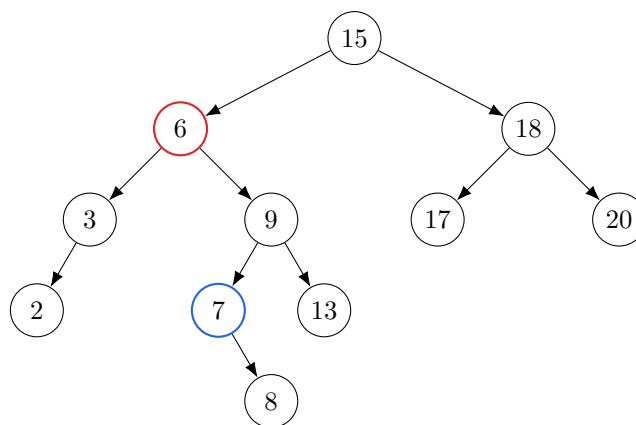


図 7: $z = 6$ を削除するとき, 右部分木の最小値 $y = 7$ を使う.

```
Node *tree_delete(Node *root, Node *z) {
    Node *y;
```

```

Node *x;
if (z->left == NULL || z->right == NULL) {
    y = z;
} else {
    y = tree_minimum(z->right);
}

if (y->left != NULL) {
    x = y->left;
} else {
    x = y->right;
}

if (x != NULL) {
    x->parent = y->parent;
}

if (y->parent == NULL) {
    root = x;
} else if (y == y->parent->left) {
    y->parent->left = x;
} else {
    y->parent->right = x;
}

if (y != z) {
    z->key = y->key;
}

free(y);
return root;
}

```

[ここをクリック](#)で映像を確認できます。

注意 3.2: 講義資料の削除コードで混乱しやすい点

z の子が 2 個のとき、削除したい key は z の key だが、実際に free される頂点は y である。つまり「 z を削除する」と言っても、物理的には y を削除し、 z の key を y の key で上書きしている。

3.8 計算量と最悪ケース

探索、最小、最大、挿入、削除はいずれも根から下へ進む操作なので、計算量は木の高さ h に比例する。

$$\text{Search, Minimum, Maximum, Insert, Delete} = O(h).$$

完全二分木に近い形なら、頂点数を n として $h = O(\log_2 n)$ である。しかし、昇順や降順の key をそのまま挿入すると、木はリストのように片側へ伸びる。この場合は $h = O(n)$ となり、二分探索木の利点が消える。

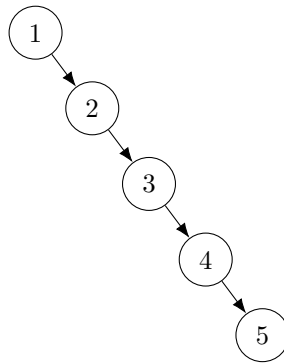


図 8: 昇順に挿入すると、右だけに伸びる最悪ケースが起きる。

3.9 平衡二分探索木と回転

最悪ケースを避けるため、木の高さを自動的に小さく保つ二分探索木が使われる。代表例は赤黒木と AVL 木である。

データ構造	高さを保つ方法
赤黒木	各頂点に赤・黒の色を持たせ、根から葉までの黒頂点数を制御する。
AVL 木	各頂点で左部分木と右部分木の高さ差を高々 1 に保つ。

どちらも、挿入や削除で崩れた形を回転 (rotation) で直す。回転は、二分探索木の大小関係を保ったまま親子関係だけを局所的に変更する操作である。

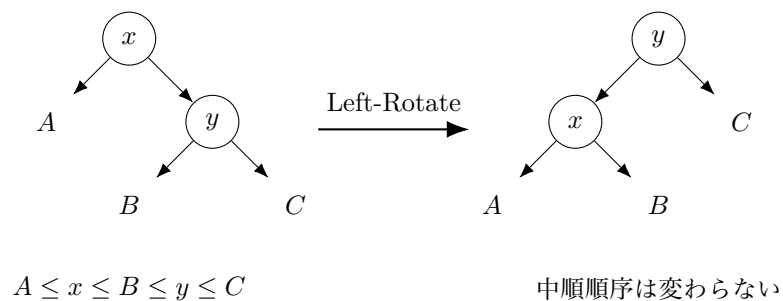


図 9: 左回転。大小関係を保ったまま高さを調整する。

3.10 中順走査: key を昇順に出力する

二分探索木では、左部分木、自分、右部分木の順にたどると key が昇順に並ぶ。

```

void inorder_tree_walk(Node *x) {
    if (x != NULL) {
        inorder_tree_walk(x->left);
        printf("%d\n", x->key);
        inorder_tree_walk(x->right);
    }
}
  
```

```
}
```

[ここをクリック](#)で映像を確認できます。

例 3.3: 図の木の中順走査

最初の例の木に対して中順走査を行うと、

2, 4, 5, 7, 8, 9, 16

の順に出力される。

3.11 削除後の確認処理

二分探索木をプログラムとして扱うときは、挿入や削除の直後に木の状態を確認すると誤りを見つけやすい。典型的には、配列 `Data` に入っている順序で `key` を探し、見つかった頂点を削除し、削除後の木に対して中順走査、最小値、最大値を出力する。重要なのは、「削除する `key`」ではなく、まず `Tree-Search` でその `key` を持つ頂点へのポインタ `z` を得ることである。

```
for (i = 0; i < N; i++) {
    z = tree_search(Tree, Data[i]);
    if (z != NULL) {
        Tree = tree_delete(Tree, z);
    }

    inorder_tree_walk(Tree);
    if (Tree != NULL) {
        min = tree_minimum(Tree);
        max = tree_maximum(Tree);
        printf("min = %d, max = %d\n", min->key, max->key);
    } else {
        printf("empty tree\n");
    }
}
```

[ここをクリック](#)で映像を確認できます。

注意 3.3: 空の木に対する最小値・最大値

すべての頂点を削除した後は、根 `Tree` が `NULL` になる。この状態で `tree_minimum(Tree)` や `tree_maximum(Tree)` を呼ぶと、`x->left` や `x->right` を読もうとして実行時エラーになる。したがって、最小値と最大値を調べる前に必ず `Tree != NULL` を確認する。

3.12 高さを再帰で求める

木の高さは、根から最も深い葉までの辺の本数である。空の木の高さを -1 と約束する。この約束を使うと、葉の高さは

$$\max(-1, -1) + 1 = 0$$

となり、辺の本数として自然に数えられる。


```

int height(Node *x) {
    int h_left;
    int h_right;

    if (x == NULL) {
        return -1;
    }

    h_left = height(x->left);
    h_right = height(x->right);
    if (h_left > h_right) {
        return h_left + 1;
    } else {
        return h_right + 1;
    }
}

```

[ここをクリック](#)で映像を確認できます。

例 3.4: 高さの計算

根だけの木では, 左右の子がともに NULL なので高さは 0 である. 根から葉まで 3 本の辺を通る最長経路がある木では, 高さは 3 である. すべての挿入が終わった直後に

```
printf("height = %d\n", height(Tree));
```

のように呼べばよい.

3.13 二色木で使う外部頂点 NILT

二色木は, 赤黒木の考え方をを用いる平衡二分探索木である. 通常の二分探索木では子が存在しないことを NULL で表したが, 二色木では外部頂点 NILT を 1 つ作り, すべての空の子がそれを指すようにする. NILT は黒として扱う.

```

NILT = (struct rbnode *)malloc(sizeof(struct rbnode));
NILT->color = 1;
NILT->parent = NILT;
NILT->left = NILT;
NILT->right = NILT;
Tree = NILT;

```

NULL と NILT の違い

- 通常の二分探索木では, 子がいないことを NULL で表す.
- 二色木では, 子がいないことを黒い外部頂点 NILT で表す.
- 二色木の停止条件は `x == NULL` ではなく, 原則として `x == NILT` である.

二色木用の最小値, 挿入, 中順走査は, 通常の二分探索木とほぼ同じである. ただし, NULL と比較していた箇

所を NILT との比較に置き換える.

```
struct rbnode *tree_minimum(struct rbnode *x) {
    while (x->left != NILT) {
        x = x->left;
    }
    return x;
}

void inorder_tree_walk(struct rbnode *x) {
    if (x != NILT) {
        inorder_tree_walk(x->left);
        printf("%d\n", x->key);
        inorder_tree_walk(x->right);
    }
}
```

3.14 黒高さ BH の確認

頂点 x の黒高さは, x から外部頂点まで降りる経路上に現れる黒頂点数を表す量である. 実装では, 子の色を足しながら左右の黒高さが等しいかを確認する. 色は

$$\text{red} = 0, \quad \text{black} = 1$$

で表す.

```
int BH(struct rbnode *x) {
    int h1;
    int h2;

    if (x == NILT) {
        return 0;
    }

    h1 = BH(x->left) + x->left->color;
    h2 = BH(x->right) + x->right->color;
    if (h1 != h2) {
        printf("BH error at key %d: left = %d, right = %d\n",
            x->key, h1, h2);
    }
    return h1;
}
```

[ここをクリック](#)で映像を確認できます.

注意 3.4: BH(Tree) と根の色

BH(Tree) は, 根そのものの色を足さず, 根の左部分木側と右部分木側を比較して返す. この定義では, left[x] と right[x] の色を足している. 根の黒さまで含めた値を出したい場合は, BH(Tree) +

Tree->color とする.

3.15 回転: 二色木の形だけを局所的に変える

二色木の挿入では, まず通常の二分探索木として赤い頂点を挿入し, その後で色と形を直す. 形を直す基本操作が左回転と右回転である. 左回転では, x の右の子 y を x の位置へ上げ, y の左部分木を x の右部分木に付け替える.

```
struct rbnode *left_rotate(struct rbnode *T, struct rbnode *x) {
    struct rbnode *r = T;
    struct rbnode *y = x->right;

    x->right = y->left;
    if (y->left != NILT) {
        y->left->parent = x;
    }

    y->parent = x->parent;
    if (x->parent == NILT) {
        r = y;
    } else if (x == x->parent->left) {
        x->parent->left = y;
    } else {
        x->parent->right = y;
    }

    y->left = x;
    x->parent = y;
    return r;
}
```

右回転は, 左右を入れ替えた操作である. x の左の子 y を x の位置へ上げ, y の右部分木を x の左部分木に付け替える.

```
struct rbnode *right_rotate(struct rbnode *T, struct rbnode *x) {
    struct rbnode *r = T;
    struct rbnode *y = x->left;

    x->left = y->right;
    if (y->right != NILT) {
        y->right->parent = x;
    }

    y->parent = x->parent;
    if (x->parent == NILT) {
        r = y;
    } else if (x == x->parent->right) {
        x->parent->right = y;
    }
}
```

```

    } else {
        x->parent->left = y;
    }

    y->right = x;
    x->parent = y;
    return r;
}

```

[ここをクリック](#)で映像を確認できます。

3.16 RB-Insert: 赤が連続したら直す

二色木への挿入は、次の 2 段階で考える。第 1 段階では、通常の二分探索木として新しい赤頂点 z を葉の位置へ入れる。第 2 段階では、 z の親も赤である間、赤頂点が親子で連続している違反を修正する。

注意 3.5: 赤黒木は赤黒が交互に並ぶ木ではない

赤黒木で禁止されるのは、赤い頂点の子が赤になることである。したがって、

黒い頂点 $\rightarrow$ 黒い子

という親子関係は許される。赤黒木は「根から葉まで赤と黒が交互に並ぶ木」ではなく、「赤赤の親子を禁止し、根から外部頂点までの黒頂点数をそろえる木」である。

```

struct rbnode *rb_insert(struct rbnode *T, struct rbnode *z) {
    T = tree_insert(T, z);
    z->color = 0;
    return rb_insert_fixup(T, z);
}

```

修正では、 z の親 p , 祖父 g , 叔父 y を見る。親が祖父の左の子である場合と右の子である場合は左右対称なので、片方を理解すればよい。

```

struct rbnode *rb_insert_fixup(struct rbnode *T, struct rbnode *z) {
    struct rbnode *y;

    while (z->parent->color == 0) {
        if (z->parent == z->parent->parent->left) {
            y = z->parent->parent->right;
            if (y->color == 0) {
                z->parent->color = 1;
                y->color = 1;
                z->parent->parent->color = 0;
                z = z->parent->parent;
            } else {
                if (z == z->parent->right) {
                    z = z->parent;
                    T = left_rotate(T, z);
                }
            }
        }
    }
}

```

```

    }
    z->parent->color = 1;
    z->parent->parent->color = 0;
    T = right_rotate(T, z->parent->parent);
}
} else {
    y = z->parent->parent->left;
    if (y->color == 0) {
        z->parent->color = 1;
        y->color = 1;
        z->parent->parent->color = 0;
        z = z->parent->parent;
    } else {
        if (z == z->parent->left) {
            z = z->parent;
            T = right_rotate(T, z);
        }
        z->parent->color = 1;
        z->parent->parent->color = 0;
        T = left_rotate(T, z->parent->parent);
    }
}
}
T->color = 1;
T->parent = NILT;
return T;
}

```

[ここをクリック](#)で映像を確認できます。

修正ケースの読み方

- 叔父 y が赤なら、親と叔父を黒にし、祖父を赤にして、違反の可能性を祖父へ押し上げる。
- 叔父 y が黒で z が内側にあるなら、まず回転して外側の形へ変える。
- 叔父 y が黒で z が外側にあるなら、親を黒、祖父を赤にして祖父を回転する。

例 3.5: 二色木挿入後の確認

二色木の挿入を検証するときは、各挿入後に中順走査と $BH(Tree)$ を実行する。

```

Tree = rb_insert(Tree, x);
inorder_tree_walk(Tree);
printf("bh = %d\n", BH(Tree));

```

中順走査が昇順であり、 BH がエラーを出さなければ、少なくとも二分探索木条件と黒高さ条件は確認できている。ただし、赤い頂点の子が赤でないことも赤黒木の条件なので、必要なら別途確認関数を作るとよい。

3.17 演習を解くためのチェックリスト

- 探索経路の問題では、「現在の頂点より小さいなら左、大きいなら右」から外れた瞬間に不可能と判断する。

- 挿入問題では, 最後に比較した頂点を y として, x を y の左または右の葉にする.
- 削除問題では, まず z の子の数を数える. 2 個なら右部分木の最小値 y を探す.
- 削除後の最小値・最大値は, 木が空でないことを確認してから呼ぶ.
- 高さは, 空の木を -1 として再帰的に求める.
- 二色木では, NULL ではなく黒い外部頂点 NILT を使う.
- BH では, 左右の黒高さが一致するかを各頂点で確認する.
- RB-Insert では, 赤い親が続く間だけ色変更と回転で直す.
- 計算量を聞かれたら, まず高さ h で答える. 平衡している仮定がある場合だけ $O(\log n)$ と言える.
- 中順走査は「左, 自分, 右」. これは二分探索木の key を昇順にするための順序である.

4 ヒープソート

4.1 この章で何を理解するか

この章では、整列問題を確認したあと、挿入ソートとヒープソートを比較する。講義資料の中心は、配列で表した 2 分ヒープを使って最大値を順に取り出し、配列を昇順へ並べ替える手続きである。ヒープソートは、ヒープを作る段階と、根を取り除いてヒープ性を修復する段階に分かれる。

この章で固定して読む記号

記号・変数	意味
A	整列したい配列. in-place 実装ではヒープも A の一部として表す.
H	ヒープを表す配列. 講義資料では説明用に A と分けて書くことがある.
n	配列の要素数. 添字は $0, 1, \dots, n-1$ とする.
i	いま注目している添字. Build-Heap では挿入位置, Heapsort では末尾側の境界を表す.
u	Upheap-Sort や Downheap-Sort で現在見ている頂点.
parent, left, right	配列添字から親, 左の子, 右の子を求める関数.

4.2 整列問題

定義 4.1: 整列問題

整列問題とは、 n 個の数値が格納された配列 A を入力として受け取り、それらを昇順に並べ替えた配列を出力する問題である。

たとえば、

15, 16, 2, 12, 3, 4, 11, 18, 1, 23

を入力とすると、出力は

1, 2, 3, 4, 11, 12, 15, 16, 18, 23

である。

代表的な整列アルゴリズム

アルゴリズム	最悪時間計算量	特徴
挿入ソート	$O(n^2)$	単純で、小さい入力では実用的に速い。
ヒープソート	$O(n \log_2 n)$	補助配列なしで実装でき、最悪計算量が良い。
マージソート	$O(n \log_2 n)$	安定で扱いやすいが、補助配列を使うことが多い。
クイックソート	$O(n^2)$	平均的には $O(n \log_2 n)$ で高速である。

4.3 挿入ソート

挿入ソートは、トランプの手札を 1 枚ずつ正しい位置へ差し込む操作と同じ考え方で動く。 i 回目の処理では、すでに

$$A[0], A[1], \dots, A[i-1]$$

が昇順に並んでいると仮定し、 $A[i]$ をその中の正しい位置へ挿入する。

```
void insertion_sort(int A[], int n) {
    for (int i = 1; i <= n - 1; i++) {
        int a = A[i];
        int j = i - 1;
        while (j >= 0 && A[j] > a) {
            A[j + 1] = A[j];
            j = j - 1;
        }
        A[j + 1] = a;
    }
}
```

[ここをクリック](#)で映像を確認できます。

挿入ソートの不変量

i 回目のループ開始時点で、左側の区間 $A[0]$ から $A[i-1]$ までは昇順に整列済みである。ループでは $a = A[i]$ より大きい要素を 1 つずつ右へずらし、空いた場所へ a を入れる。これにより、次のループ開始時点では $A[0]$ から $A[i]$ ままで整列済みになる。

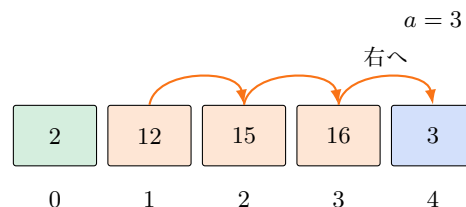
例 4.1: 挿入ソートの 1 回の処理

$$A[0], A[1], A[2], A[3], A[4] = 2, 12, 15, 16, 3$$

で $i = 4$ とする。 $a = 3$ を入れるために 16, 15, 12 を右へずらし、最後に 2 の右隣へ 3 を入れるので、

$$2, 3, 12, 15, 16$$

となる。



2 は 3 以下なので、3 は添字 1 へ入る

図 10: 挿入ソートで $a = 3$ を挿入する様子。 a より大きい 12, 15, 16 を右へずらし、空いた位置へ a を入れる。

i 回目では最大で i 個の要素を右へずらす。したがって総時間は

$$O(n) + \sum_{i=1}^{n-1} O(i) = O(n^2)$$

である。

4.4 完全二分木と効率的な二分木

ヒープを理解するためには、まず高さの低い二分木を考える。完全二分木では、すべての葉の深さが同じで、葉でない頂点は子を 2 個持つ。高さを d とすると、頂点数 n は

$$n = 2^0 + 2^1 + \dots + 2^d = 2^{d+1} - 1$$

である。したがって、

$$d = \log_2(n+1) - 1$$

となり、頂点数が 2 倍になっても高さはほぼ 1 しか増えない。

定義 4.2: 効率的な二分木

効率的な二分木とは、完全二分木の最下段を左詰めで部分的に残した形の二分木である。高さは頂点数 n に対して $O(\log_2 n)$ である。

4.5 2 分ヒープ

定義 4.3: 2 分ヒープ

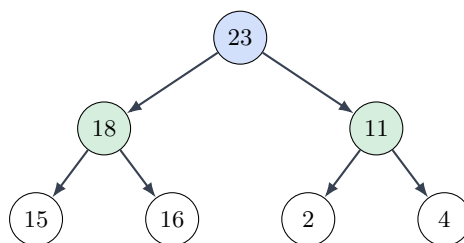
2 分ヒープとは、効率的な二分木の各頂点に値を格納し、親の値が子の値以上であるというヒープ性を満たすデータ構造である。

$$H[\text{parent}(i)] \geq H[i]$$

を各頂点 i について満たすとき、根には最大値が入る。

配列添字を 0 始まりにすると、親と子の添字は次で求められる。

$$\text{parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor, \quad \text{left}(i) = 2i+1, \quad \text{right}(i) = 2i+2$$



0:23, 1:18, 2:11, 3:15, 4:16, 5:2, 6:4

図 11: 配列で表した最大ヒープ。親子関係は添字から計算できるため、ポインタは不要である。

ヒープ性は「親が子以上である」ことだけを要求する。したがって、同じ深さの頂点どうしや、左右の部分木全体の大小関係は要求しない。この点が二分探索木との重要な違いである。

4.6 Upheap-Sort

新しい値をヒープ末尾へ追加した直後は、その頂点と親の間だけでヒープ性が壊れている可能性がある。そのため、その頂点から根へ向かって、親より大きい間だけ値を交換する。この修復手続きが Upheap-Sort である。

```
void upheap_sort(int H[], int i) {
    int u = i;
    while (u > 0 && H[u] > H[parent(u)]) {
        swap(H[u], H[parent(u)]);
        u = parent(u);
    }
}
```

[ここをクリック](#)で映像を確認できます。

Upheap-Sort で壊れうる場所

挿入前の $H[0]$ から $H[i-1]$ まではヒープである。末尾 i に値を入れると、新しくできた辺 ($\text{parent}(i), i$) だけがヒープ性を壊す可能性を持つ。交換後も壊れうるのは、交換で上がった値とその新しい親の間だけである。

根までのパスの長さはヒープの高さ以下なので、Upheap-Sort の計算量は

$$O(\log_2 i)$$

である。

4.7 Insert と Build-Heap

Insert は、末尾へ値を置いてから Upheap-Sort を呼び出す手続きである。Build-Heap は、空のヒープへ $A[0], A[1], \dots, A[n-1]$ を順に挿入してヒープを作る。

```
void insert(int H[], int i, int a) {
    H[i] = a;
    upheap_sort(H, i);
}

void build_heap(int H[], int A[], int n) {
    for (int i = 0; i <= n - 1; i++) {
        insert(H, i, A[i]);
    }
}
```

[ここをクリック](#)で映像を確認できます。

例 4.2: Build-Heap の途中状態

入力を

15, 16, 2, 12, 3, 4, 11, 18, 1, 23

とする。すべて挿入すると、講義資料の例では

$H = 23, 18, 11, 15, 16, 2, 4, 12, 1, 3$

が得られる。根 $H[0]$ には最大値 23 がある。

Insert の計算量は $O(\log_2 i)$ である。Build-Heap ではこれを n 回行うので、

$$\sum_{i=1}^n O(\log_2 i) = O(n \log_2 n)$$

である。

4.8 根を取り除く操作と Downheap-Sort

最大ヒープでは根に最大値が入っている。根を取り除いたあと、最後の頂点の値を根へ移動すると、根とその子の間だけでヒープ性が壊れる可能性がある。そこで、根から葉へ向かって、子のうち大きい方と必要に応じて交換する。この修復手続きが Downheap-Sort である。

```
void downheap_sort(int H[], int i) {
    int u = 0;
    while (true) {
        int l = (left(u) <= i) ? left(u) : u;
        int r = (right(u) <= i) ? right(u) : u;
        int largest = u;

        if (H[l] > H[largest]) {
            largest = l;
        }
        if (H[r] > H[largest]) {
            largest = r;
        }
        if (largest == u) {
            break;
        }
        swap(H[u], H[largest]);
        u = largest;
    }
}
```

[ここをクリック](#)で映像を確認できます。

Downheap-Sort の比較対象

現在地 u に左の子がなければ $l = u$, 右の子がなければ $r = u$ と見なす. そのうえで u, l, r の中で最大値を持つ添字を選ぶ. 最大が u 自身ならヒープ性は回復済みであり, そうでなければ最大の子と交換して下へ進む.

交換は根から葉までの 1 本のパス上でしか起きない. したがって Downheap-Sort の計算量は

$$O(\log_2 i)$$

である.

4.9 Heapsort

ヒープソートは, まず配列全体から最大ヒープを作り, その後で最大値を末尾へ順に移していく. 最大値を末尾に置いたあとの区間は整列済みとみなし, 残った未整列区間だけをヒープとして扱う.

```
void heapsort(int A[], int n) {
    build_heap(A, A, n);
    for (int i = 1; i <= n - 1; i++) {
        swap(A[0], A[n - i]);
        downheap_sort(A, n - i - 1);
    }
}
```

[ここをクリック](#)で映像を確認できます.

Heapsort の区間の見方

ループ中の配列は 2 つの区間に分かれる.

$A[0], \dots, A[n - i - 1]$ 未整列だがヒープとして扱う区間

$A[n - i], \dots, A[n - 1]$ 最大値から順に置かれた整列済み区間

根 $A[0]$ を末尾側へ交換するたびに, 整列済み区間が 1 要素広がる.

$A[0]$ は未整列区間の最大値

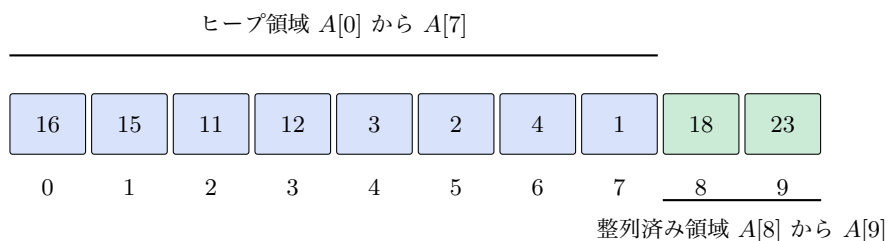


図 12: ヒープソート中の配列の見方. 左側を最大ヒープとして保ち, 右側には取り出した最大値を昇順になる位置へ固定していく.

Build-Heap は $O(n \log_2 n)$ である。その後、Downheap-Sort を高々 $n - 1$ 回呼び出し、各回は $O(\log_2 n)$ である。したがって全体の計算量は

$$O(n \log_2 n) + O(n \log_2 n) = O(n \log_2 n)$$

である。

4.10 補助配列を用いない実装

講義資料では説明のために H と A を分けているが、実装では配列 A だけで十分である。Build-Heap 中は、 $A[0]$ から $A[i]$ までをヒープ領域、 $A[i+1]$ から $A[n-1]$ までを未挿入領域と見ればよい。最大値を取り出す段階では、 $A[0]$ から $A[n-i-1]$ までをヒープ領域、 $A[n-i]$ から $A[n-1]$ までを整列済み領域と見ればよい。

注意 4.1: in-place であることの意味

ヒープソートは、配列内の要素交換だけで整列を進められる。そのため、入力配列と同じ大きさの補助配列を必要としない。この性質を sort-in-place という。

4.11 ヒープの応用

2 分ヒープは、最大値または最小値を繰り返し取り出す用途に向いている。根以外の任意の頂点を効率よく削除する構造ではないが、高さの調整が不要で扱いやすい。この性質は優先度付きキューに直結する。

ヒープで重要な操作

- 挿入: 新しい値を末尾へ入れ、Upheap-Sort で直す。
- 根の削除: 最大値または最小値を取り出し、Downheap-Sort で直す。
- 値の変更: 値が大きくなれば上へ、小さくなれば下へ動かして直す。

より高度なヒープとして Fibonacci heap がある。ならし解析を用いると、挿入 n 回、根の削除 m 回、値の変更 p 回を含む操作列に対して

$$O(n + m \log n + p)$$

のような評価が現れる。最短路アルゴリズムなどでは、このような優先度付きキューの性能が全体の計算量に影響する。

5 ハッシュ法

5.1 この章で何を理解するか

この章では、辞書的なデータ管理を高速に行うためのハッシュ法を学ぶ。探索木を使うと探索は $O(\log n)$ で行えるが、辞書やコンパイラの記号表では、できれば探索、挿入、削除を平均的に $O(1)$ で行いたい。ハッシュ法は、key から配列のアドレスを計算することで、その理想に近づくための方法である。

この章で固定して読む記号

記号・変数	意味
S	管理したい項目の集合。辞書なら単語集合、コンパイラなら変数名や関数名の集合。
n	現在格納されている項目数。
m	ハッシュ表の大きさ。アドレスは $0, 1, \dots, m-1$ 。
A	ハッシュ表を表す 1 次元配列。
L	外部ハッシュ法で各セルを管理するリスト用配列。
$h(x)$	key x をハッシュ表のアドレスへ写すハッシュ関数。
$h_i(x)$	内部ハッシュ法で衝突後に調べる i 番目のアドレス。
NIL	セルやアドレスが存在しないことを表す番兵値。配列実装では -1 を使うことが多い。

5.2 辞書的なデータ管理

辞書的なデータ管理では、主に次の 3 つの操作を扱う。

- 挿入: 新しい key をデータ構造へ加える。
- 探索: 与えられた key が含まれるかを調べ、含まれるなら関連情報を取り出す。
- 削除: 既に格納されている key を取り除く。

電子辞書やコンパイラの記号表では、特に探索が何度も行われる。平衡二分探索木を使えば探索は $O(\log n)$ でできるが、ハッシュ法では平均的に $O(1)$ を目標にする。

例 5.1: 探索問題の例

英単語の辞書なら、 S は英単語の集合であり、探索は「recreational の意味を調べる」操作である。コンパイラなら、 S はそれまでに登場した変数名や関数名の集合であり、新しい名前が既に使われていないかを調べる操作になる。

5.3 直接配列で持つ方法とその限界

key が取り得る値の種類が有限なら、key を整数に対応させ、その整数を配列の添字として使う方法が考えられる。たとえば文字列 x を構成する文字コードを並べたり足し合わせたりすれば、 x から整数を作れる。

直接アドレス指定の考え方

各項目 x_i に整数 k_i を対応させ、十分大きな配列 A を用意する.

$$x_i \in S \iff A[k_i] \text{ が空でない}$$

とすれば、探索、挿入、削除はいずれも $O(1)$ で行える.

しかし、この方法は配列が巨大になりすぎる. たとえば 8 文字以下の変数名を 1 文字 8 ビットで表すと、最大で 64 ビット、つまり約 $2^{64} \simeq 10^{18}$ 種類の値があり得る. 実際にプログラム内で使われる変数名はそのごく一部なので、ほとんどの配列要素が空のままになる.

注意 5.1: ハッシュ法の動機

直接配列の「key からアドレスを計算すれば速い」という発想は有用である. ただし、あり得る全 key に対応する巨大配列を用意するのではなく、実際に使う項目数に見合う大きさの配列へ圧縮して写す必要がある. これがハッシュ法の出発点である.

5.4 ハッシュ表とハッシュ関数

定義 5.1: ハッシュ表

ハッシュ表とは、項目を格納するための大きさ m の 1 次元配列である. アドレスは $0, 1, \dots, m-1$ で表す.

定義 5.2: ハッシュ関数

ハッシュ関数とは、各 key x に対して、その項目を格納する候補アドレスを定める関数である.

$$0 \leq h(x) \leq m-1$$

を満たす整数値関数として考える.

ハッシュ関数は、key をできるだけ不規則にハッシュ表全体へ散らすものが望ましい. 偏りが大きいと、特定のアドレスに項目が集中し、探索が遅くなる.

例 5.2: ハッシュ関数の例

文字列 $x = a_1 a_2 \dots a_w$ に対して、

$$K(x) = \sum_{i=1}^w \text{ord}(a_i)$$

とする. このとき、

$$h(x) = K(x) \bmod m$$

とすれば、 x を 0 から $m-1$ までのアドレスへ写せる. 割り算法では、 m は 2 のべき乗に近くない素数を選ぶとよい.

別の例として、 $0 < B < 1$ を満たす定数 B を用い、

$$h(x) = \lfloor m \cdot (K(x)B \text{ の小数部分}) \rfloor$$

とする乗算法もある. また、ハッシュ関数の集合を用意し、key ごとにランダムに選ぶ 2 段階ハッシングも考えられる.

5.5 衝突と 2 つの解決法

ハッシュ法では、異なる項目 x_i, x_j が同じアドレスへ写ることがある。

$$h(x_i) = h(x_j)$$

となる現象を衝突という。ハッシュ表の大きさ m は、key の全種類数よりずっと小さいので、衝突を完全に避けることはできない。

衝突への代表的な対処

- 外部ハッシュ法: 同じハッシュ値を持つ項目を、そのアドレスから始まる線形リストにつなぐ。
- 内部ハッシュ法: ポインタを使わず、衝突した項目もハッシュ表の別アドレスへ格納する。

外部ハッシュ法は chaining または open hashing と呼ばれる。内部ハッシュ法は open addressing または closed hashing と呼ばれる。

5.6 外部ハッシュ法

外部ハッシュ法では、ハッシュ表 A の各要素が線形リストの先頭を指す。同じハッシュ値を持つ項目は、そのリストに連結して格納する。

定義 5.3: 外部ハッシュ法

外部ハッシュ法とは、ハッシュ表の各アドレスにバケットを対応させ、同じバケットに入る項目を線形リストとして管理する方法である。

key a を探索するときは、まず $h(a)$ を計算し、 $A[h(a)]$ が指すリストだけを調べればよい。ハッシュ関数が十分ランダムなら、1 つのリストに入る項目数の期待値は n/m である。特に $m > n$ なら探索の期待時間は $O(1)$ になる。

注意 5.2: 外部ハッシュ法の計算量

平均的には高速だが、最悪の場合にはすべての項目が同じハッシュ値を持つ可能性がある。その場合、1 本の長いリストを探索することになり、計算量は $O(n)$ である。

5.7 外部ハッシュ法の探索

外部ハッシュ法の探索は、前章のリスト探索とほぼ同じである。

```
int chained_hash_search(Table A, List L, string a) {
    int x = A[h(a)];
    while (x != NIL && L.key[x] != a) {
        x = L.next[x];
    }
    return x;
}
```


例 5.3: 探索の例

$m = 6$ とし, 文字コードの和を m で割った余りをハッシュ値にする.

$$h(\text{"sum"}) = 341 \bmod 6 = 5$$

なので, "sum" を探すときは $A[5]$ が指すリストだけをたどる. そのリスト内で見つければセルのアドレスを返し, 末尾まで行っても見つからなければ NIL を返す.

5.8 外部ハッシュ法の挿入

挿入では, 新しいセル x の key が既に設定されているとする. 挿入位置は $A[h(\text{key}[x])]$ が指すリストの先頭にする.

```
void chained_hash_insert(Table A, List L, int x) {
    int j = h(L.key[x]);
    L.next[x] = A[j];
    A[j] = x;
}
```

リストの先頭に差し込むだけなので, 挿入自体は $O(1)$ である. ただし, 同じ key を重複して入れない前提なら, 挿入前に探索して「まだ存在しない」ことを確認する.

```
int y = chained_hash_search(A, L, a);
if (y == NIL) {
    int x = allocate_object(L);
    L.key[x] = a;
    chained_hash_insert(A, L, x);
}
```

5.9 外部ハッシュ法の削除

削除では, 対象セル x が属するリストから x を外す. 単方向リストで実装している場合, x の直前セル z を探す必要がある.

```
void chained_hash_delete(Table A, List L, int x) {
    int j = h(L.key[x]);
    int z = NIL;
    int y = A[j];

    while (y != NIL && y != x) {
        z = y;
        y = L.next[y];
    }

    if (y == NIL) {
        return;
    }
}
```

```

if (z != NIL) {
    L.next[z] = L.next[x];
} else {
    A[j] = L.next[x];
}
}
}

```

削除で分ける場合

- x がリストの先頭でない: 直前セル z の `next` を `next[x]` に張り替える.
- x がリストの先頭である: ハッシュ表 $A[h(\text{key}[x])]$ を `next[x]` に更新する.

指定された key a を削除したいときは, まず探索でセル x を見つける. 見つかった場合だけリストから外し, その後 `free_object` などでセルを未使用リストへ戻す.

5.10 配列による外部ハッシュ法の実装

C 言語などで外部ハッシュ法を配列実装する場合, 各セルには key と次セルのアドレスを持たせる.

```

struct cell {
    char key[W + 1];
    int next;
};

struct cell L[n];
int A[m];

```

ここで W は文字列長の上限, n は項目数の上限, m はハッシュ表の大きさである. 文字列の終端文字 `'\0'` を入れるため, key 配列は長さ $W + 1$ とする.

注意 5.3: C の文字列操作

文字列の比較には `strcmp(a, b)`, 文字列のコピーには `strcpy(a, b)` を使う. また, 文字列の各文字を整数に変換して足し合わせれば, 簡単なハッシュ値のもとになる整数を作れる.

5.11 内部ハッシュ法

内部ハッシュ法では, すべての項目をハッシュ表 A の中に直接格納する. そのため, 少なくとも $m \geq n$ でなければならない.

定義 5.4: 内部ハッシュ法

内部ハッシュ法とは, 衝突が起きたときに, 別の候補アドレス

$$h_1(x), h_2(x), \dots, h_{m-1}(x)$$

を順に調べ, 空いているセルに項目を格納する方法である.

項目 x を挿入するとき, $h(x)$ が空ならそこへ入れる. 既に使用中なら, $h_1(x), h_2(x), \dots$ を順に調べ, 最初

に見つかった空きセルへ入れる。

5.12 内部ハッシュ法の探索

探索では、挿入時に調べた可能性のあるアドレスを同じ順番で調べる。

内部ハッシュ法の探索規則

- 空きセルを見つける前に key a が見つかったなら、そのアドレスを返す。
- key a をつける前に未使用セルが現れたなら、 a は表にないと判断する。
- m 個の候補を調べ尽くした場合も、見つからなければ NIL を返す。

この規則は「もし a が表にあるなら、挿入時に未使用セルを飛び越えて奥へ入ることはない」という事実に基づく。

5.13 線形走査法

内部ハッシュ法で最も単純な候補アドレスの作り方が線形走査法である。

定義 5.5: 線形走査法

線形走査法とは、

$$h_i(x) = (h(x) + i) \bmod m$$

として、衝突したら次のアドレス、さらに次のアドレス、という順で空きを探す方法である。

たとえば $m = 11$ で $h(\text{"Tree"}) = 4$ のとき、アドレス 4, 5, 6 が使用済みなら、次の候補である 7 に "Tree" を格納する。探索でも同じ順で調べる。

例 5.4: 線形走査で存在しないと判断する例

$h(\text{"head"}) = 6$ とする。アドレス 6 から順に調べ、"head" をつける前に未使用セルへ到達したなら、"head" はハッシュ表にないと判断できる。

5.14 内部ハッシュ法における削除の注意

内部ハッシュ法では、削除したセルを単純に空に戻すと探索が壊れることがある。ある項目 a が衝突を避けて後ろのセルに入っていた場合、その手前のセルを空にしまうと、次に a を探したときにそこで探索が止まってしまうからである。

例 5.5: 単純な削除による誤判定

"Tree" が $h(\text{"Tree"}) = 4$ から始まる走査の結果、アドレス 7 に格納されていたとする。その途中にあるアドレス 5 の "List" を単純に空セルへ戻すと、次に "Tree" を探すときアドレス 5 で探索が止まり、"Tree" は存在しないと誤判定される。

この問題を避けるため、各セルに状態 `state` を持たせる。

状態	意味
Empty	まだ一度も使われていないセル. 探索はここで止めてよい.
Occupied	現在 key が格納されているセル.
Deleted	以前使われたが削除済みのセル. 探索では使用中と同じように通過する.

注意 5.4: Deleted の役割

Deleted は, 挿入では再利用できる空きセルとして扱う. しかし探索では, その先に目的の key が入っている可能性があるため, Empty のように探索を止めてはいけない.

5.15 内部ハッシュ法の挿入

線形走査法で挿入する擬似コードは次のように書ける. 挿入前には, 同じ key がまだ表にないことを探索で確認しておく.

```
void hash_insert(Table A, string a) {
    int base = h(a);
    int x = NIL;
    int i = 0;

    while (x == NIL && i < m) {
        int y = (base + i) % m;
        if (A.state[y] != Occupied) {
            x = y;
        } else {
            i = i + 1;
        }
    }

    if (x == NIL) {
        output("error: out of space");
    } else {
        A.key[x] = a;
        A.state[x] = Occupied;
    }
}
```

最悪の場合はハッシュ表全体を調べるので, 計算量は $O(m)$ である.

5.16 内部ハッシュ法の探索と削除

```
int hash_search(Table A, string a) {
    int base = h(a);
    int i = 0;

    while (i < m) {
        int x = (base + i) % m;
        if (A.state[x] == Occupied && A.key[x] == a) {
```

```

        return x;
    }
    if (A.state[x] == Empty) {
        return NIL;
    }
    i = i + 1;
}
return NIL;
}

```

```

void hash_delete(Table A, string a) {
    int x = hash_search(A, a);
    if (x != NIL) {
        A.key[x] = NIL;
        A.state[x] = Deleted;
    }
}

```

探索と削除も、最悪の場合は m 個のセルを調べるので $O(m)$ である。平均的には、ハッシュ関数が十分ランダムで表に余裕があれば高速に動く。

5.17 内部ハッシュ法の平均的な計算量

内部ハッシュ法の平均計算量を考えるときは、各項目 x について調べるアドレス列

$$h(x), h_1(x), h_2(x), \dots, h_{m-1}(x)$$

が $0, 1, \dots, m-1$ のランダムな順列のように振る舞うと仮定する。負荷率を

$$\alpha = \frac{n}{m}$$

とおく。

平均的な見積もり

- 表にない key の探索では、調べるセル数の期待値はおおよそ $\frac{1}{1-\alpha}$ 。
- 表にある key の探索では、その key を挿入したときに調べたセル数と同程度。
- $m \simeq 2n$ なら $\alpha \simeq 1/2$ なので、平均的には十分高速である。

つまり、ハッシュ表を詰め込みすぎないことが重要である。表が満杯に近づくほど空きセルを見つけにくくなり、挿入や失敗探索が急に重くなる。

5.18 線形走査法の問題点

線形走査法は実装が簡単だが、使用中セルが連続して固まる傾向がある。これを primary clustering という。

注意 5.5: primary clustering

使用中セルの連続区間が長くなると、その直前に衝突した項目は連続区間の末尾まで進むことになる。さらに、長い連続区間ほど次の挿入を吸収しやすいため、ますます伸びやすい。その結果、ランダムな場合に期待される性能より悪くなる。

講義では、 $m/2$ 個の項目が入っている場合でも、使用中セルが 1 個おきに散らばっていれば失敗探索の平均は小さいが、前半 $m/2$ 個が連続して埋まっていると平均調査数が大きくなることを例にしている。

5.19 他の走査法

線形走査法の固まりやすさを避けるため、他の候補アドレス列を使うことがある。

定義 5.6: 2 次走査法

2 次走査法とは、

$$h_i(x) = (h(x) + c_1 i + c_2 i^2) \bmod m$$

のように、 i の 2 次式で候補アドレスを作る方法である。

2 次走査法は線形走査法よりよい性能を持つことが多い。ただし、すべてのアドレスをうまく走査できるようにするには、 c_1, c_2, m の選び方に条件がある。

定義 5.7: ダブルハッシュ法

ダブルハッシュ法とは、2 つのハッシュ関数 h, g を用いて、

$$h_i(x) = (h(x) + i g(x)) \bmod m$$

と候補アドレスを作る方法である。

ダブルハッシュ法は非常によい性能を持つ。ただし、すべてのアドレスを巡回できるように、 $g(x)$ と m が常に互いに素になるように設計する必要がある。

5.20 補足: 一般的な二分探索

講義の最後では、二分探索木での探索をより一般的な二分探索の特殊ケースとして見ている。二分探索は、求めたい値 x^* が存在する範囲 $[L, U]$ が分かっており、単調性により左右どちらに絞ればよいか判断できるときに使える。

```
real binary_search(real L, real U, real epsilon) {
    while (abs(U - L) >= epsilon) {
        real M = (L + U) / 2;
        if (answer_is_in_left_side(L, M)) {
            U = M;
        } else {
            L = M;
        }
    }
    return L;
}
```

}

例 5.6: 単調関数の零点探索

連続で単調増加な関数 $f(x)$ について, $f(x^*) = 0$ を満たす x^* を求めたいとする. $f(L) \leq 0, f(U) \geq 0$ が分かっているならば, 中点 $M = (L + U)/2$ を調べ,

$$f(M) \geq 0 \Rightarrow U \leftarrow M, \quad f(M) < 0 \Rightarrow L \leftarrow M$$

として範囲を半分にできる. 幅が ε 未満になるまで繰り返せばよい.

5.21 演習を解くためのチェックリスト

- key をそのまま巨大配列の添字にする方法と, ハッシュ表へ圧縮して写す方法の違いを説明できるようにする.
- ハッシュ表の大きさ m , 項目数 n , 負荷率 $\alpha = n/m$ を区別する.
- 衝突が起きたとき, 外部ハッシュ法では「同じバケットのリスト」へ入ることを確認する.
- 外部ハッシュ法の探索, 挿入, 削除はリスト操作に帰着する.
- 内部ハッシュ法では, 挿入時と探索時に同じ候補アドレス列をたどる.
- 内部ハッシュ法の削除では, Empty と Deleted を区別する.
- 線形走査法では $h_i(x) = (h(x) + i) \bmod m$ を使う.
- 線形走査法の primary clustering が, 平均性能を悪化させる理由を言葉で説明できるようにする.
- 2 次走査法とダブルハッシュ法は, 衝突後の候補アドレス列を工夫する方法である.

6 バケット法

6.1 この章で何を理解するか

この章では、値の存在範囲があらかじめ分かっているデータを、いくつかの領域に分けて管理するバケット法を学ぶ。講義資料では、平面上の最近点探索を例にしてバケット法の考え方を説明し、その応用としてバケットソートと基数ソートへ進む。重要なのは、データが範囲内にほぼ一様に分布しているなら、1つのバケットに入る要素数を平均的に定数個へ抑えられるという点である。

この章で固定して読む記号

記号・変数	意味
n	データ数. 最近点探索では点の個数, ソートでは配列の要素数.
k	空間や桁を分ける基準. 最近点探索では各軸を k 個に分ける. 基数ソートでは k 進数の基数.
m	バケット数. バケットソートでは $m = n$ とすることが多いが, n の定数倍でもよい.
A	整列したい配列. 添字は $0, 1, \dots, n-1$ とする.
B	各バケットの先頭セルを指す配列.
L	バケット内の線形リストを配列で表すためのセル配列.
l, u	ソート対象の値が入る区間 $[l, u]$ の下端と上端.
d	基数ソートで扱う最大桁数.
a_p	k 進表現における k^p の桁の係数.
NIL	リストの終端や空バケットを表す番兵値. 配列実装では -1 を使うことが多い.

6.2 バケット法の考え方

定義 6.1: バケット法

バケット法とは、データの存在する範囲を複数の領域に分け、各データを属する領域のバケットに格納して管理する方法である。各バケットの中身は、通常は線形リストで管理する。

バケット法は、次の条件があるときに有効である。

- データが取り得る値の範囲が既知である。
- データがその範囲内でほぼ一様に分布している。
- 目的の操作で調べるべきバケット数が定数個程度に抑えられる。

たとえば平面上の領域 $[0, 1] \times [0, 1]$ に n 個の点

$$P_1, P_2, \dots, P_n$$

が分布しているとする。 k を $\sqrt{n}$ に近い整数とし、領域を幅 $1/k$ 、高さ $1/k$ の正方形に区切る。このときバケット数は $k^2 \simeq n$ であり、点が一様に分布していれば 1つのバケットに入る点数の平均は約 1 個になる。

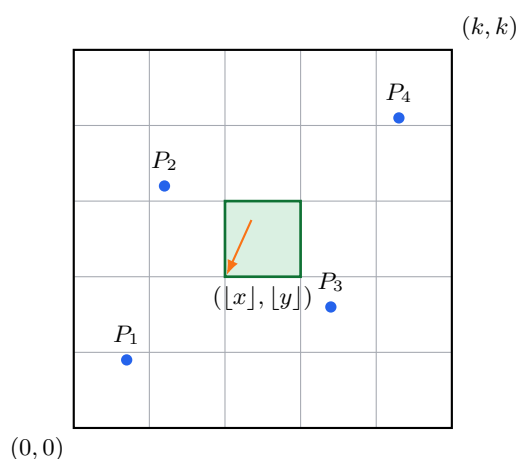


図 13: $[0, k] \times [0, k]$ を k^2 個の正方形バケットに分ける. 点 $P(x, y)$ は左下隅が $(\lfloor x \rfloor, \lfloor y \rfloor)$ のバケットに入る.

座標を整数で扱う見方

講義資料では説明を簡単にするため, すべての座標を k 倍して考えている. すると領域は $[0, k] \times [0, k]$ となり, 点 $P(x, y)$ が属するバケットは

$$(\lfloor x \rfloor, \lfloor y \rfloor)$$

で表せる. たとえば $P(3.2, 5.8)$ は, 左下隅が $(3, 5)$ のバケットに属する.

6.3 最近点探索問題

定義 6.2: 最近点探索問題

既知の点 $P_1, P_2, \dots, P_n$ が与えられているとする. 新しく与えられた点 $P(x, y)$ に対し, $P_1, \dots, P_n$ の中で P に最も近い点を求める問題を最近点探索問題という.

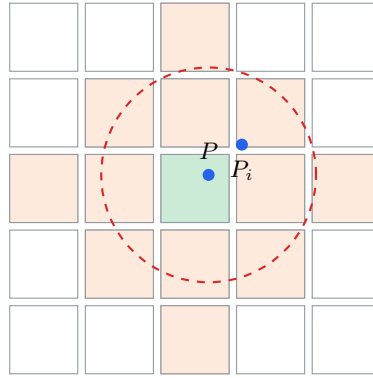
最近点探索は, 地図上で現在地に最も近い施設を探す場合などに現れる. 全点との距離を毎回計算すると $O(n)$ かかるが, 点が一様に分布しているならバケット法で平均的に高速化できる.

6.4 同じバケットに点がある場合

まず, 問い合わせ点 P の属するバケットに, 既知の点 P_i が入っている場合を考える. バケットの一辺の長さは 1 なので, 同じバケットに入っている 2 点の距離は高々 $\sqrt{2}$ である. したがって, 最近点は P の属するバケットから距離 $\sqrt{2}$ 以内のバケットの中に必ず存在する.

調べる範囲

同じバケットに点があるからといって, その点 P_i が最近点とは限らない. 隣接する別のバケットに, もっと近い点があるかもしれない. そのため, P の属するバケットから距離 $\sqrt{2}$ 以内にあるバケットをすべて調べる. 講義資料では, この場合に調べるべきバケット数は 21 個としている.



斜線円の近くにある候補バケットだけを調べる

図 14: 同じバケットに点がある場合. 暫定距離は高々 $\sqrt{2}$ なので, 周辺の定数個のバケットを調べればよい.

バケット数が $k^2 \simeq n$ で十分大きく, 点が一様に分布していれば, 21 個のバケットに含まれる点の期待数は定数個である. したがって最近点候補の確認は平均的に $O(1)$ で行える.

6.5 同じバケットに点がない場合

次に, P の属するバケットが空の場合を考える. このときは, P に近いバケットから順に調べ, 初めて空でないバケットを見つける. そのバケットに入っている点を P_i とし,

$$d = \text{dist}(P, P_i)$$

とおく.

空バケットから探索する手順

- P の属するバケットから近い順に, 空でないバケットを探す.
- 見つかったバケット内の点 P_i から, 暫定距離 $d = \text{dist}(P, P_i)$ を得る.
- P の属するバケットから距離 d 以内にあるバケットをすべて調べる.
- その範囲内の点と P の距離を計算し, 最小のものを最近点として返す.

暫定距離 d より遠いバケットに入る点は, 最近点になる可能性がない. したがって, 初めて見つかった点で探索範囲を決め, その範囲だけを調べればよい.

6.6 最近点探索の計算量

点数 n が十分大きく, 点が領域内に一様に分布していると仮定する. $k^2 \simeq n$ となるようにバケットを作ると, 1 つのバケットに入る点数の平均は約 1 個である. また, 問い合わせ時に調べるバケット数も平均的には定数個と見なせる.

注意 6.1: 平均 $O(1)$ の意味

バケット法の $O(1)$ は, どんな入力でも必ず定数時間という意味ではない. 点が一箇所に偏れば, 1 つのバケットに多くの点が入り, そのリストを調べる時間は大きくなる. 一様分布という仮定のもとで, 平均的に定数時間になるという意味である.

最近点探索に限らず、ある範囲に一様に分布する対象をバケットで管理できるなら、探索や範囲内の候補確認を平均的に $O(1)$ で行えることがある。

6.7 バケットソート

バケット法は整列にも利用できる。整列問題では、 n 個の数値が格納された配列 A を昇順に並べ替える。たとえば

15, 16, 2, 12, 3, 4, 11, 18, 1, 23

を入力とすると、出力は

1, 2, 3, 4, 11, 12, 15, 16, 18, 23

である。

定義 6.3: バケットソート

バケットソートとは、値の範囲 $[l, u]$ を複数の区間に分け、各値に対応するバケットへ入れたあと、各バケット内を整列し、バケットの順に連結して全体を昇順にする整列法である。

講義資料では、 n 個の実数が区間 $[l, u]$ に一様に分布している場合を考える。区間 $[l, u]$ を n 個のバケットに等分すると、1 つのバケットに入る値の平均個数は約 1 個になる。そのため、各バケット内の整列は平均的に小さい入力 of 整列になる。

6.8 バケット番号の計算

n 個のバケットを使う場合、値 a が属するバケット番号 j は、次の不等式を満たす番号である。

$$j \frac{u-l}{n} \leq a-l < (j+1) \frac{u-l}{n}$$

したがって、

$$j \leq \frac{n(a-l)}{u-l} < j+1$$

なので、

$$j = \left\lfloor \frac{n(a-l)}{u-l} \right\rfloor$$

と計算できる。

注意 6.2: 上端 u の扱い

式をそのまま使うと $a = u$ のとき $j = n$ になる。実装では、入力が $l \leq a < u$ を満たすと仮定するか、 $a = u$ のときだけ最後のバケット $n-1$ に入れるなどの処理を入れる。

バケット数を m 個にする場合は、

$$j = \left\lfloor \frac{m(a-l)}{u-l} \right\rfloor$$

へ変更すればよい。 m が n の定数倍なら、1 バケットあたりの平均要素数は定数個のみである。

6.9 Bucket-Sort

講義資料の Bucket-Sort は次の形で書ける.

```
void bucket_sort(real A[], int n, real l, real u) {
    for (int i = 0; i <= n - 1; i++) {
        int j = floor(n * (A[i] - l) / (u - l));
        insert A[i] at the head of bucket[j];
    }

    for (int j = 0; j <= n - 1; j++) {
        insertion_sort(bucket[j]);
    }

    concatenate bucket[0], bucket[1], ..., bucket[n - 1] into A;
}
```

平均計算量

最初のバケット挿入は $O(n)$, バケットを順に見る処理も $O(n)$ である. 各バケットに入る値の個数が平均的に定数なら, バケット 1 個を挿入ソートする時間も平均的に $O(1)$ である. したがって, バケットソート全体の平均計算量は

$$O(n)$$

になる.

挿入ソートは最悪 $O(r^2)$ だが, r が小さいときは単純で高速である. バケットソートでは, バケット内の要素数が小さくなるように範囲を分けるため, 挿入ソートとの相性がよい.

6.10 バケットソートの例

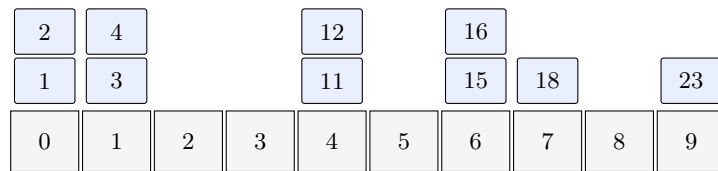
$$A = 15, 16, 2, 12, 3, 4, 11, 18, 1, 23$$

に対して, $n = 10$, $l = 0$, $u = 25$ とする. このときバケット幅は

$$\frac{u - l}{n} = \frac{25}{10} = 2.5$$

である.

バケット番号	入る値
0	1, 2
1	3, 4
2	なし
3	なし
4	11, 12
5	なし
6	15, 16
7	18
8	なし
9	23



バケット番号の小さい順に連結
 $A: 1, 2, 3, 4, 11, 12, 15, 16, 18, 23$

図 15: $n = 10, l = 0, u = 25$ のバケットソート例. 各バケット内を整理し, 左から順に連結すると全体が昇順になる.

各バケット内を昇順に整理し, バケット番号の小さい順に連結すると,

1, 2, 3, 4, 11, 12, 15, 16, 18, 23

が得られる.

6.11 配列によるバケットの実装

バケット内のリストは, ポインタではなく配列で実装できる. 講義資料では, バケット配列 B とセル配列 L を使う.

```
struct cell {
    real key;
    int next;
};

cell L[n];
int B[m];
```

初期状態では, セル i に $A[i]$ を対応させる.

$$L[i].key = A[i], \quad L[i].next = \text{NIL}$$

とし、すべてのバケットについて

$$B[j] = \text{NIL}$$

としておく.

```
void bucket_insert(int B[], cell L[], int x, int j) {
    L[x].next = B[j];
    B[j] = x;
}
```

配列実装で変化するもの

セルの **key** は最初に $A[i]$ からコピーした値のままでよい. バケットへ挿入するときに変化するの、各セルの **next** と、バケット配列 B の先頭アドレスだけである. 必要なリスト操作は「先頭に挿入」だけなので、実装は単純である.

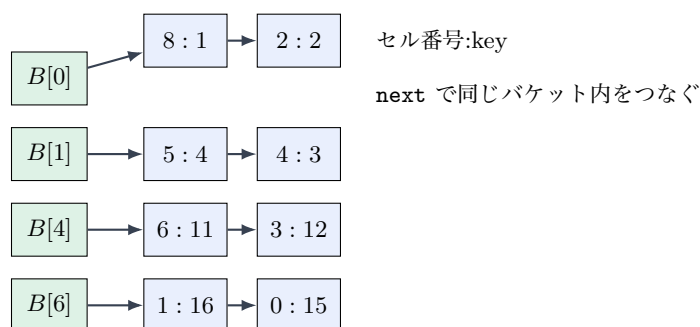


図 16: 配列によるバケットリストの見方. $B[j]$ はバケット j の先頭セルを指し、各セルの **next** が次のセルを指す.

バケットの中身を配列 A に戻すときは、バケット $0, 1, \dots, m-1$ を順にたどる. バケット j までに格納した要素数を t_j とおくと、バケット j の値が A に入る領域は

$$[t_{j-1}, t_j - 1]$$

である. ただし $t_{-1} = 0$ とする. バケットが空なら $t_{j-1} = t_j$ なので、そのバケットに対応する領域は存在しない.

6.12 基数ソート

定義 6.4: 基数ソート

基数ソートとは、数値を k 進数の桁列として見て、低い桁から順に各桁で安定ソートを行う整列法である.

ソートすべき n 個の数値が、いずれも d 桁以下の k 進数で表されたとする. 人間が番号を整理するときは、最高位の桁でグループ分けし、各グループ内をさらに整列することが多い. しかし、これをそのまま計算機で再帰的に実装すると、グループ管理のために大きな記憶領域が必要になりやすい.

基数ソートでは、逆に低い桁から処理する. 同じ桁の値を持つデータの相対順序を保つ安定ソートを使うことで、低い桁で作った順序を壊さずに、だんだん高い桁まで正しい順序を作れる.

定義 6.5: 安定ソート

安定ソートとは、等しい key を持つ要素どうしの相対的な順序を変えない整列法である。

```
void radix_sort(int A[], int n, int k, int d) {
    for (int p = 0; p <= d - 1; p++) {
        stable_sort A by the digit of k^p;
    }
}
```

各桁の安定ソートにバケットソートや分布数え上げソートを使う。各桁のソートにかかる時間を $T(n, k)$ とすれば、全体の計算量は

$$O(dT(n, k))$$

である。

6.13 バケット法を用いた基数ソート

k 進数の 1 桁は $0, 1, \dots, k-1$ のいずれかである。したがって、各桁の値に対応する k 個のバケットを用意すればよい。バケット内の値はすべて同じ桁の値を持つので、バケット内をさらにソートする必要はない。

安定にするための挿入順

講義資料では、配列を後ろから前へ処理し、各要素に対応するバケットのリスト先頭へ挿入する。後ろから見て先頭挿入することで、同じ桁の値を持つ要素の相対順序が元の配列と同じになる。つまり、このバケット処理は安定ソートとして働く。

```
void radix_sort_with_buckets(int A[], int n, int k, int d) {
    for (int p = 0; p <= d - 1; p++) {
        for (int i = 0; i <= k - 1; i++) {
            clear bucket[i];
        }

        for (int j = n - 1; j >= 0; j--) {
            int i = digit(A[j], k, p);
            insert A[j] at the head of bucket[i];
        }

        concatenate bucket[0], bucket[1], ..., bucket[k - 1] into A;
    }
}
```

各桁について、バケット初期化に $O(k)$ 、配列要素の挿入に $O(n)$ 、バケットの連結に $O(k + n)$ 程度かかる。したがって全体の計算量は

$$O(d(k + n))$$

である。

6.14 基数ソートの例

10 進数

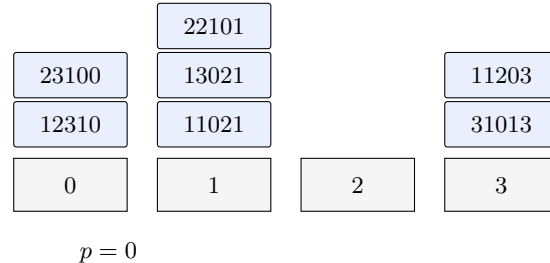
329, 457, 657, 839, 436, 720, 355

を考える. これらは 4 進数では次のように表せる.

10進表記	4進表記
329	11021_4
457	13021_4
657	22101_4
839	31013_4
436	12310_4
720	23100_4
355	11203_4

低い桁から順に安定ソートすると, 配列は次のように変化する.

処理後	配列
$p = 0$	12310, 23100, 11021, 13021, 22101, 31013, 11203
$p = 1$	23100, 22101, 11203, 12310, 31013, 11021, 13021
$p = 2$	31013, 11021, 13021, 23100, 22101, 11203, 12310
$p = 3$	31013, 11021, 11203, 22101, 12310, 13021, 23100
$p = 4$	11021, 11203, 12310, 13021, 22101, 23100, 31013



$p = 0$

concat

$A: 12310, 23100, 11021, 13021, 22101, 31013, 11203$

図 17: 基数ソートの 1 桁目 ($p = 0$) の処理. 後ろから見て先頭挿入するため, 同じ桁の要素の相対順序が保たれる.

最後に 10 進表記へ戻すと,

329, 355, 436, 457, 657, 720, 839

となり, 昇順に整列されている.

6.15 k 進表現の桁の計算

自然数 x の k 進表現を

$$x = (a_{d-1}, \dots, a_1, a_0)_k$$

と書く。これは

$$x = a_{d-1}k^{d-1} + \cdots + a_1k + a_0$$

を意味する。ここで a_p は k^p の桁の係数であり、

$$0 \leq a_p \leq k-1$$

を満たす。

k^p の桁を直接求めるには、

$$q = x \operatorname{div} k^p$$

とおき、

$$a_p = q \bmod k$$

とすればよい。つまり、

$$a_p = \left(\left\lfloor \frac{x}{k^p} \right\rfloor \right) \bmod k$$

である。

例 6.1: k 進表現の桁の計算

$x = 355$, $k = 4$, $p = 2$ とする。

$$355 \operatorname{div} 4^2 = 22, \quad 22 \bmod 4 = 2$$

なので、 4^2 の桁は $a_2 = 2$ である。実際、

$$355 = 11203_4$$

であり、低い桁から数えて $p = 2$ の桁は 2 である。

低い桁から順に計算する場合は、毎回 k^p で割り直すより、商を更新していくほうが効率的である。

```
void compute_digits(int x, int k, int d, int a[]) {
    int y = x;
    for (int p = 0; p <= d - 1; p++) {
        int q = y div k;
        a[p] = y - q * k;    // y mod k
        y = q;
    }
}
```

除算は比較的成本が高いので、実装では除算回数を減らす工夫が重要になる。

6.16 演習を解くためのチェックリスト

- バケット法が有効になる条件として、値の範囲が既知で、データが一様に分布していることを説明できるようにする。
- 最近点探索では、問い合わせ点が属するバケットが空かどうかで処理が分かれることを確認する。
- $k^2 \simeq n$ とすると、平面上の各バケットに入る点数の平均が約 1 個になることを説明できるようにする。
- バケットソートで、値 a のバケット番号が $\lfloor n(a-l)/(u-l) \rfloor$ になる理由を不等式から導けるようにする。
- バケット数を m 個に変える場合は、 $\lfloor m(a-l)/(u-l) \rfloor$ を使う。

- 配列実装では, $B[j]$ がバケット j の先頭セルを指し, $L[x].\text{next}$ でリストをつなぐ.
- 基数ソートでは, 低い桁から順に安定ソートする理由を説明できるようにする.
- バケットを使う基数ソートで, 配列を後ろから処理してリスト先頭へ挿入すると安定になることを確認する.
- k^p の桁の係数は $(\lfloor x/k^p \rfloor) \bmod k$ で求められる.

7 分布数え上げソートと比較ソートの下限

7.1 この章で何を理解するか

この章では、値の範囲が整数として限られているときに使える分布数え上げソートを学ぶ。続いて、値の大小比較だけで動く比較ソートには $O(n \log_2 n)$ の下限があることを決定木で見る。最後に、次章へつながる題材としてハノイの塔を再帰呼び出しで表す。

この章で固定して読む記号

記号・変数	意味
A	整列したい入力配列。添字は $0, 1, \dots, n-1$ 。
B	分布数え上げソートで使う補助配列。整列後の値を一時的に入れる。
C	累積個数を表す配列。 $C[i]$ は「値が i 以下の要素数」を表す。
n	入力配列の要素数。
k	入力値の種類数。分布数え上げソートでは値が $0, \dots, k-1$ に入る。
d	基数ソートで扱う最大桁数。
h	比較ソートの決定木の高さ。最悪比較回数の下界になる。

7.2 数値の性質を使う整列

比較だけで整列する方法とは別に、数値が持つ追加情報を使う整列法がある。第6回で扱ったバケットソートと基数ソートもその例である。

整列法	入力への仮定	計算量
バケットソート	$[l, u]$ に一様に分布する実数	平均 $O(n)$
基数ソート	d 桁以下の k 進数	$O(d(n+k))$
分布数え上げソート	$0, \dots, k-1$ の整数	$O(n+k)$

分布数え上げソートは、値そのものを添字として使えるため、比較ソートの下限を回避できる。ただし、 k が非常に大きいと補助配列 C が大きくなりすぎる。

7.3 分布数え上げソートの考え方

定義 7.1: 分布数え上げソート

分布数え上げソートとは、入力値が 0 以上 $k-1$ 以下の整数であることを利用し、各値の出現数と累積個数から整列後の位置を直接決める整列法である。

配列 C を次の意味で使う。

$$C[i] = \#\{j \mid A[j] \leq i\}$$

つまり $C[i]$ は「値が i 以下の要素数」である。値 i を持つ要素は、整列後の配列 B の前から $C[i]$ 番目までのどこかに入る。添字が 0 始まりなので、最後の位置は $C[i]-1$ である。

配置の基本手順

- $i = A[j]$ とする.
- $A[j]$ を $B[C[i] - 1]$ に入れる.
- 次に同じ値 i が来たときは 1 つ前へ入れるため, $C[i] \leftarrow C[i] - 1$ とする.

A	5	3	2	6	4	1	3	7	2	0
	0	1	2	3	4	5	6	7	8	9
B										7

$C[7] = 10$ なので, 値 7 は $B[9]$ に入る

図 18: $C[i]$ を使った配置. 値 i は $B[C[i] - 1]$ に置き, その後 $C[i]$ を 1 減らす.

7.4 安定性

入力配列 A を後ろから見ると, 同じ値を持つ要素の相対順序が変わらない. この性質を安定性という.

定義 7.2: 安定ソート

安定ソートとは, 等しい key を持つ要素どうしの入力中の前後関係を, 出力でも保つ整列法である.

分布数え上げソートを基数ソートの各桁のソートとして使うには, この安定性が不可欠である. 低い桁で作った順序を, 高い桁の処理で壊してはいけないからである.

7.5 Counting-Sort

```
void counting_sort(int A[], int n, int k) {
    int C[0..k-1];
    int B[0..n-1];

    for (int i = 0; i <= k - 1; i++) {
        C[i] = 0;
    }
    for (int j = 0; j <= n - 1; j++) {
        C[A[j]] = C[A[j]] + 1;
    }
    for (int i = 1; i <= k - 1; i++) {
        C[i] = C[i - 1] + C[i];
    }

    for (int j = n - 1; j >= 0; j--) {
        B[C[A[j]] - 1] = A[j];
        C[A[j]] = C[A[j]] - 1;
    }
    for (int j = 0; j <= n - 1; j++) {
        A[j] = B[j];
    }
}
```

計算量

- C の初期化: $O(k)$.
- 出現数の計算: $O(n)$.
- 累積和の計算: $O(k)$.
- B への配置と A へのコピー: $O(n)$.

したがって全体の計算量は

$$O(n + k)$$

である.

7.6 $C[i]$ の値を効率よく求める

$C[i]$ を直接「 $A[j] = i$ となる j の数」として各 i について全探索すると、 $O(kn)$ かかる。効率よく行うには、配列 A を 1 回だけ走査して出現数を増やす。

```
for (int i = 0; i <= k - 1; i++) {
    C[i] = 0;
}
for (int j = 0; j <= n - 1; j++) {
    C[A[j]] = C[A[j]] + 1;
}
for (int i = 1; i <= k - 1; i++) {
    C[i] = C[i - 1] + C[i];
}
```

最後のループで、出現数の配列が累積個数の配列へ変わる。

7.7 分布数え上げソートによる基数ソート

基数ソートの各桁の安定ソートとして Counting-Sort を使うと、

$$O(d(n + k))$$

で d 桁以下の k 進数を整列できる。

```
void radix_sort_with_counting(int A[], int n, int k, int d) {
    for (int p = 0; p <= d - 1; p++) {
        counting_sort_by_digit(A, n, k, p);
    }
}
```

ここで `counting_sort_by_digit` は、 $A[j]$ そのものではなく $A[j]$ の k^p の桁の値を key として Counting-Sort を行う手続きである。

7.8 比較ソート法

定義 7.3: 比較ソート法

比較ソート法とは、入力値について特別な範囲や分布を仮定せず、値どうしの 2 値比較だけで整列する方法である。

挿入ソート、ヒープソート、マージソート、クイックソートは比較ソートである。比較ソートでは、入力の並び方を比較結果の列だけで区別しなければならない。この制約から、最悪の場合に必要な比較回数の下限が生まれる。

7.9 決定木モデル

比較ソートの動作は決定木で表せる。内部頂点は「 a_i と a_j のどちらが小さいか」という比較であり、葉は出力される順列である。

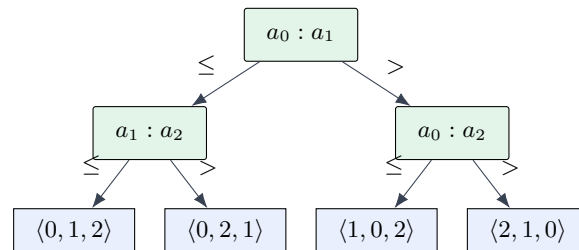


図 19: 比較ソートの決定木の見方。根から葉までのパスが、その入力で行われる比較列に対応する。

n 個の値がすべて異なると仮定すると、入力の順序は $n!$ 通りある。正しく整列するには、決定木に少なくとも $n!$ 個の葉が必要である。高さ h の 2 分木の葉数は高々 2^h なので、

$$2^h \geq n!, \quad h \geq \log_2(n!)$$

が成り立つ。

Stirling の公式を使うと、

$$\log_2(n!) = \Omega(n \log_2 n)$$

である。したがって、任意の比較ソートには最悪の場合

$$\Omega(n \log_2 n)$$

回の比較が必要である。

注意 7.1: 比較ソートの最適性

ヒープソートやマージソートは最悪 $O(n \log_2 n)$ で動く。比較ソートの下限が $\Omega(n \log_2 n)$ なので、これらは比較ソートとしては漸近的に最適である。

7.10 ハノイの塔と再帰

ハノイの塔は、 n 枚の円盤を柱 A から柱 C へ移す問題である。一度に動かせる円盤は 1 枚だけで、小さい円盤の上に大きい円盤を置いてはいけない。

```
void move(int n, Pole A, Pole B, Pole C) {
    if (n == 1) {
        move disk from A to C;
    } else {
        move(n - 1, A, C, B);
        move largest disk from A to C;
        move(n - 1, B, A, C);
    }
}
```

移動回数を $T(n)$ とすると,

$$T(1) = 1, \quad T(n) = 2T(n-1) + 1$$

である. これを解くと

$$T(n) = 2^n - 1$$

となる. 指数時間だが, ハノイの塔ではそれ自体が避けられない本質的な手間である.

7.11 演習を解くためのチェックリスト

- $C[i]$ が「値が i 以下の要素数」を表すことを説明できるようにする.
- Counting-Sort で A を後ろから見る理由が安定性のためであることを確認する.
- C の初期化, 出現数計算, 累積和計算を分けて書けるようにする.
- 分布数え上げソートの計算量が $O(n+k)$ になる理由を説明できるようにする.
- 比較ソートの決定木で, 葉が少なくとも $n!$ 個必要になる理由を押さえる.
- $h \geq \log_2(n!) = \Omega(n \log_2 n)$ が比較ソートの下限を与えることを確認する.
- ハノイの塔の再帰式 $T(n) = 2T(n-1) + 1$ から $T(n) = 2^n - 1$ を導けるようにする.

8 再帰呼び出しと分割統治法

8.1 この章で何を理解するか

この章では、再帰呼び出しを使って問題を小さな問題へ分ける考え方を学ぶ。題材はハノイの塔、クイックソート、マージソート、2つの数の積の計算である。分割統治法では、問題を分割し、子問題を再帰的に解き、必要なら解を統合する。

この章で固定して読む記号

記号・変数	意味
$A[p..r]$	ソート対象の部分配列。
x	Quicksort の基準値。講義資料では $x = A[p]$ 。
q	Partition が返す分割位置。
$T(n)$	入力サイズ n の問題を解く時間。
a, b	再帰式 $T(n) = aT(n/b) + g(n)$ で、子問題数と縮小率を表す。
p, q, r, s	Multiply で x, y を上位半分と下位半分に分けた値。

8.2 分割統治法

定義 8.1: 分割統治法

分割統治法とは、問題をいくつかの子問題に分け、子問題を再帰的に解き、その解を統合して元の問題の解を作る方法である。

3つの段階

- 分割: 問題を複数の子問題へ分ける。
- 統治: 子問題を再帰的に解く。
- 統合: 子問題の解から元の問題の解を構成する。

再帰が効率的になるためには、子問題のサイズが元の問題から定比率で小さくなることが重要である。ハノイの塔では $n-1$ にしか減らないため指数時間になるが、マージソートや高速な積の計算では $n/2$ へ減るため効率がよい。

8.3 Quicksort

Quicksort は、基準値を使って配列を左右に分け、左右を再帰的にソートする。分割後は、左側の各要素が右側の各要素以下になるので、統合の作業は不要である。

```
void quicksort(int A[], int p, int r) {
    if (p < r) {
        int q = partition(A, p, r);
        quicksort(A, p, q);
        quicksort(A, q + 1, r);
    }
}
```

```
}
```

$$x = A[p] = 15$$

15	16	2	12	3	4	11	18	1	23
----	----	---	----	---	---	----	----	---	----

1	11	2	12	3	4	16	18	15	23
---	----	---	----	---	---	----	----	----	----

≤ 15 ≥ 15

図 20: Partition の結果. 基準値 $x = 15$ に対して, 左側は x 以下, 右側は x 以上になる.

8.4 Partition

Partition は $A[p..r]$ を基準値 $x = A[p]$ で分ける. 右から x 以下の値を探し, 左から x 以上の値を探して, まだ交差していなければ交換する.

```
int partition(int A[], int p, int r) {
    int x = A[p];
    int i = p;
    int j = r;

    while (true) {
        while (A[j] > x) {
            j = j - 1;
        }
        while (A[i] < x) {
            i = i + 1;
        }
        if (i < j) {
            swap(A[i], A[j]);
            i = i + 1;
            j = j - 1;
        } else {
            return j;
        }
    }
}
```

この手続きでは各要素を高々定数回見るだけなので,

$$O(r - p + 1)$$

で動く.

注意 8.1: 停止条件の置き方

講義資料では, `while (i < j)` を外側の条件にするのではなく, 無限ループの中で $i \geq j$ になったときに抜ける書き方を推奨している. 外側の条件だけで止めると, $i = j$ で止まったときに分割位置が正しくない例が出るためである.

8.5 Quicksort の計算量

Partition が返す分割が極端に偏ると, 再帰式は

$$T(n) = T(1) + T(n-1) + cn$$

となり,

$$T(n) = O(n^2)$$

である. すでに昇順に整列された配列で, 先頭要素を基準にし続けるとこの状況が起こる.

一方, 常に半分ずつに分かれるなら,

$$T(n) = 2T(n/2) + cn$$

となり,

$$T(n) = O(n \log_2 n)$$

である.

一定比率なら十分

分割比が 1 : 9 のように悪く見えても, 毎回一定比率で小さくなるなら再帰木の高さは $O(\log n)$ である. 各深さでの Partition の合計は $O(n)$ なので, 全体は $O(n \log n)$ になる.

平均的には $O(n \log_2 n)$ であり, 実用上は高速である. 入力順に依存しにくくするため, 基準値をランダムに選ぶ Randomized-Quicksort が使われる.

8.6 Merge-Sort

Merge-Sort は, 配列を半分に分け, それぞれを再帰的にソートし, 最後に 2 つの整列済み配列を統合する.

```
void merge_sort(int A[], int p, int r) {
    if (p < r) {
        int q = floor((p + r) / 2);
        merge_sort(A, p, q);
        merge_sort(A, q + 1, r);
        merge(A, p, q, r);
    }
}
```

8.7 Merge

```
void merge(int A[], int p, int q, int r) {
    int i = p;
```

```

int j = q + 1;
int k = p;

while (i <= q && j <= r) {
    if (A[i] <= A[j]) {
        B[k] = A[i];
        i = i + 1;
    } else {
        B[k] = A[j];
        j = j + 1;
    }
    k = k + 1;
}
while (i <= q) {
    B[k] = A[i]; i = i + 1; k = k + 1;
}
while (j <= r) {
    B[k] = A[j]; j = j + 1; k = k + 1;
}
for (int t = p; t <= r; t++) {
    A[t] = B[t];
}
}

```

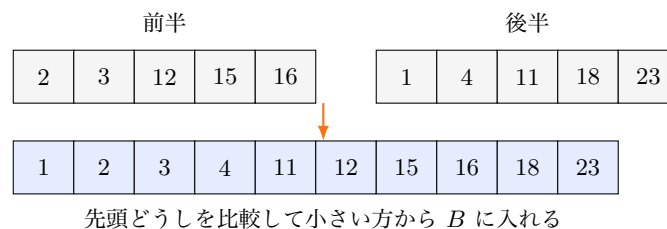


図 21: Merge の考え方. 2 つの整列済み列を先頭から比較して 1 つの整列済み列へ統合する.

Merge は $A[p..r]$ の長さに比例する時間で動く. したがって Merge-Sort の再帰式は

$$T(n) = 2T(n/2) + cn$$

であり,

$$T(n) = O(n \log_2 n)$$

である.

8.8 再帰式の一般形

大きさ n の問題を, 大きさ n/b の子問題 a 個へ分け, 統合に $g(n)$ かかるとする. このとき

$$T(n) = aT(n/b) + g(n)$$

という再帰式が現れる。特に $g(n) = cn$ のとき,

$$\gamma = \log_b a$$

とおくと, 大まかに次のように読める.

条件	計算量
$a > b$	$O(n^{\log_b a})$
$a = b$	$O(n \log n)$
$a < b$	$O(n)$

Merge-Sort は $a = b = 2$ なので $O(n \log n)$ である.

8.9 2つの数の積の計算

n 桁の 2 進数 x, y を半分に分ける.

$$x = p2^{n/2} + q, \quad y = r2^{n/2} + s$$

とすると,

$$xy = pr2^n + (ps + qr)2^{n/2} + qs$$

である. 素直には pr, ps, qr, qs の 4 回の半サイズ乗算が必要である.

しかし,

$$ps + qr = (p + q)(r + s) - pr - qs$$

なので, 半サイズ乗算は

$$pr, \quad qs, \quad (p + q)(r + s)$$

の 3 回で済む.

```
int multiply(int x, int y, int n) {
    if (n == 1) {
        return x * y;
    }
    split x into p and q;
    split y into r and s;
    int u = multiply(p + q, r + s, n / 2);
    int v = multiply(p, r, n / 2);
    int w = multiply(q, s, n / 2);
    return v * 2^n + (u - v - w) * 2^(n / 2) + w;
}
```

再帰式は

$$T(n) = 3T(n/2) + cn$$

であり,

$$T(n) = O(n^{\log_2 3}) \simeq O(n^{1.58})$$

となる. これは素朴な $O(n^2)$ より小さい.

8.10 演習を解くためのチェックリスト

- Partition は $A[p..q] \leq x$, $A[q+1..r] \geq x$ を満たす q を返す.
- Quicksort の最悪分割は 1 と $n-1$, 最良分割は $n/2$ と $n/2$ である.
- Merge では, 前半と後半の先頭を比較し, 小さい方を補助配列へ入れる.
- Merge-Sort の再帰式 $T(n) = 2T(n/2) + cn$ を解けるようにする.
- 一般再帰式 $T(n) = aT(n/b) + g(n)$ の $a, b, g(n)$ を読み取る.
- 高速乗算では, $ps + qr$ を $(p+q)(r+s) - pr - qs$ で求める.

9 グラフ探索

9.1 この章で何を理解するか

この章では、グラフの基本用語と表現方法を確認し、グラフ探索の枠組みを学ぶ。探索途中の頂点集合をキューで管理すると幅優先探索、スタックまたは再帰で管理すると深さ優先探索になる。どちらも多くのグラフアルゴリズムの基礎になる。

この章で固定して読む記号

記号・変数	意味
$G = (V, E)$	グラフ. V は頂点集合, E は辺集合.
$ V , E $	頂点数と辺数.
$d(v)$	BFS では始点から v までの距離. DFS では発見時刻として使う.
$f(v)$	DFS で v の探索が終了した時刻.
$\pi(v)$	探索木における v の親.
white	未発見の頂点.
gray	発見済みだが探索未完了の頂点.
black	探索完了した頂点.

9.2 グラフと隣接

定義 9.1: グラフ

グラフとは、頂点集合 V と辺集合 E の組 $G = (V, E)$ である。無向グラフでは辺 $\{u, v\}$ と $\{v, u\}$ を区別せず、有向グラフでは辺 (u, v) と (v, u) を区別する。

2 つの頂点が辺でつながれているとき、それらは隣接しているという。有向グラフでは、辺 (u, v) は u から出て v へ入る辺である。

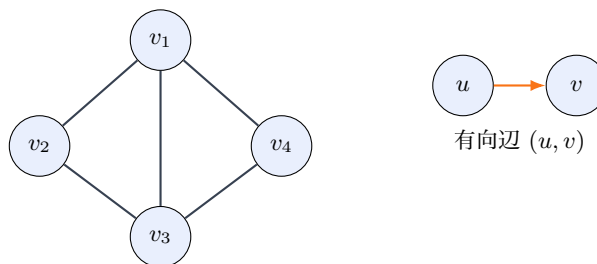


図 22: 無向グラフと有向辺. 無向辺は向きを持たず、有向辺は出発点と到着点を区別する。

9.3 グラフの表現

陽に与えられたグラフは、主に隣接行列または隣接リストで表す。

定義 9.2: 隣接行列

頂点数 n のグラフに対し, $n \times n$ 行列 A を用意し, 頂点 v_i と v_j が隣接するなら $a_{ij} = 1$, そうでなければ $a_{ij} = 0$ とする表現を隣接行列という.

定義 9.3: 隣接リスト

各頂点ごとに, その頂点に隣接する頂点のリストを持つ表現を隣接リストという.

表現による違い

隣接行列では, ある頂点 v の隣接頂点を調べるために v の行をすべて見るので $O(|V|)$ かかる. 隣接リストでは, 実際に隣接している頂点だけをたどるため, v の次数に比例する.

無向グラフでは, 次数の総和は辺数の 2 倍である.

$$\sum_{v \in V} \deg(v) = 2|E|$$

したがって隣接リスト全体の長さは $O(|V| + |E|)$ である.

9.4 グラフ探索の基本形

始点 v_0 から到達可能な頂点をすべて見つけたい. 基本形では, 発見済みで未探索の頂点集合 A と, 発見済み頂点集合 B を管理する.

```
Graph_Search(G, v0) {
    make A and B empty;
    insert v0 into A and B;
    while (A is not empty) {
        remove one vertex v from A;
        for each vertex u adjacent to v {
            if (u is not in B) {
                insert u into A and B;
            }
        }
    }
    output B;
}
```

A の取り出し方を変えると探索順が変わる. キューで管理すれば幅優先探索, スタックで管理すれば深さ優先探索になる.

9.5 キューとスタック

キューは先入れ先出し, スタックは後入れ先出しである.

構造	方針	グラフ探索
キュー	FIFO	幅優先探索
スタック	LIFO	深さ優先探索

どちらも 1 次元配列で実装でき、挿入と削除は $O(1)$ で行える。キューでは `head` と `tail` を循環させ、スタックでは `top` を末尾位置として管理する。

9.6 幅優先探索

定義 9.4: 幅優先探索

幅優先探索とは、始点から近い頂点を先に探索するグラフ探索である。発見済み未探索の頂点をキューで管理する。

```
void BFS(Graph G, Vertex s) {
    for each v in V - {s} {
        color[v] = white;
        d[v] = INF;
        pi[v] = NIL;
    }
    color[s] = gray;
    d[s] = 0;
    pi[s] = NIL;
    make queue Q empty;
    enqueue(Q, s);

    while (Q is not empty) {
        Vertex v = dequeue(Q);
        for each u adjacent to v {
            if (color[u] == white) {
                color[u] = gray;
                d[u] = d[v] + 1;
                pi[u] = v;
                enqueue(Q, u);
            }
        }
        color[v] = black;
    }
}
```

BFS の性質

- 始点 s から到達可能な頂点はすべて発見される。
- $d[v]$ は s から v への最短パスの辺数になる。
- $\pi[v]$ を親として集めると、 s を根とする幅優先木が得られる。
- 各頂点は高々 1 回だけキューへ入る。

9.7 BFS の計算量

隣接行列を使う場合、各頂点を取り出すたびに行全体を調べるので、

$$O(|V|^2)$$

である。

隣接リストを使う場合、各頂点は高々 1 回取り出され、各辺は無向グラフでは高々 2 回調べられる。したがって、

$$O(|V| + |E|)$$

である。隣接リストなら線形時間で探索できる。

9.8 深さ優先探索

定義 9.5: 深さ優先探索

深さ優先探索とは、最後に発見した頂点から出る辺を優先して探索し、行き止まりになったら後戻りする探索である。

DFS はスタックでも実装できるが、再帰を使うと簡潔である。発見時刻 $d[v]$ と終了時刻 $f[v]$ を記録する。

```
void DFS(Graph G, Vertex s) {
    for each v in V {
        color[v] = white;
        d[v] = INF;
        f[v] = INF;
        pi[v] = NIL;
    }
    time = 0;
    traverse(s);
}

void traverse(Vertex v) {
    color[v] = gray;
    time = time + 1;
    d[v] = time;

    for each u adjacent to v {
        if (color[u] == white) {
            pi[u] = v;
            traverse(u);
        }
    }

    color[v] = black;
    time = time + 1;
    f[v] = time;
}
```

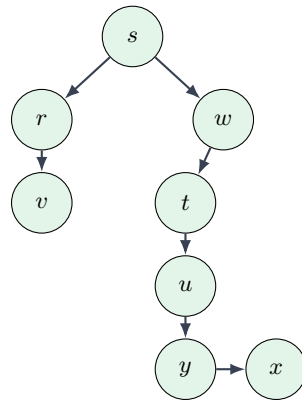


図 23: 深さ優先木の例. 先に深く進み, 未探索辺がなくなると後戻りする.

9.9 DFS の性質と計算量

DFS も, 隣接リストなら

$$O(|V| + |E|)$$

であり, 隣接行列なら

$$O(|V|^2)$$

である.

発見時刻と終了時刻

任意の 2 頂点 u, v について, 区間 $[d[u], f[u]]$ と $[d[v], f[v]]$ は, 互いに交わらないか, 一方が他方を含むかのどちらかである. 含まれる場合, 含まれた側の頂点は DFS 木で相手の子孫である.

9.10 すべての頂点を訪問する DFS

グラフが連結でない場合, 1 つの始点からでは全頂点に到達できない. そのため, すべての頂点を順に見て, 未発見なら `traverse` を呼ぶ.

```
void DFS_all(Graph G) {
    for each v in V {
        color[v] = white;
        d[v] = INF;
        f[v] = INF;
        pi[v] = NIL;
    }
    time = 0;
    for each v in V {
        if (color[v] == white) {
            traverse(v);
        }
    }
}
```

このとき得られる親辺の集合は 1 本の木ではなく、複数の木からなる深さ優先森になる。

9.11 DFS の応用

DFS は多くの問題の基礎になる。

- 無向グラフの連結成分分解.
- 切断点の列挙.
- 有向グラフの強連結成分分解.

陽に与えられていないグラフでは、頂点に色を直接持たせられない場合がある。その場合は、局面や状態を整数・文字列に符号化し、ハッシュ表などで「発見済み」を管理する。

9.12 演習を解くためのチェックリスト

- 隣接行列と隣接リストの違いを、隣接頂点を調べる時間で説明できるようにする。
- BFS はキュー、DFS はスタックまたは再帰を使う。
- BFS の $d[v]$ は始点からの最短辺数である。
- DFS の $d[v]$ と $f[v]$ は発見時刻と終了時刻である。
- 隣接リストなら BFS/DFS は $O(|V| + |E|)$ で動く。
- 連結でないグラフ全体を DFS するには、未発見頂点ごとに Traverse を呼ぶ。

10 最短路問題

10.1 この章で何を理解するか

この章では、重みつきグラフにおける最短路問題と Dijkstra 法を学ぶ。Dijkstra 法は、すべての辺の重みが非負であるとき、始点から各頂点への最短距離を小さい順に確定していく算法である。後半では、候補集合を 1 次元配列や 2 分ヒープで管理したときの計算量を比較する。

この章で固定して読む記号

記号・変数	意味
$G = (V, E)$	重みつきグラフ。無向でも有向でもよい。
$l(u, v)$	辺 (u, v) の長さまたは重み。Dijkstra 法では非負。
v_s, v_t	始点と終点。
$d(v)$	現時点で分かっている v_s から v への最短距離候補。
$d^*(v)$	真の最短距離。
$\pi(v)$	最短路上で v の直前にある頂点。
S	最短距離が確定した頂点集合。
A	次に確定する候補頂点の集合。
H	候補集合 A を管理する最小ヒープ。

10.2 最短路問題

定義 10.1: 最短路問題

重みつきグラフ $G = (V, E)$ と始点 v_s 、終点 v_t が与えられたとき、 v_s から v_t へ到達するパスのうち、辺の重みの総和が最小のものを求める問題を最短路問題という。

鉄道経路検索やカーナビでは、駅や交差点を頂点、線路や道路を辺として最短路問題を解いている。Dijkstra 法はすべての辺長が非負の場合に使える。負の辺を許す場合には Bellman-Ford 法などが必要になる。

10.3 最短距離を確定する性質

S を、始点 v_s からの最短距離がすでに確定した頂点集合とする。 S に入っていない頂点 w について、

$$d(w) = \min\{d^*(y) + l(y, w) \mid y \in S\}$$

を候補距離とする。この候補値が最小の頂点 v は、真の最短距離がすでに分かったと考えてよい。すなわち

$$d(v) = d^*(v)$$

である。

Dijkstra 法の要点

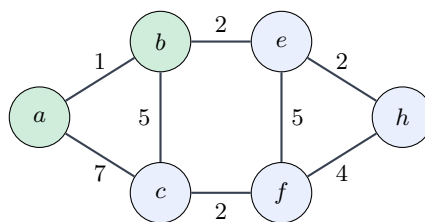
最短距離が確定した集合 S から 1 本の辺で出ていける頂点だけを候補にする。候補距離が最小の頂点を S に加えると、非負辺重みのもとでは後からより短い経路が現れることはない。

10.4 Dijkstra 法

```
void dijkstra(Graph G, Vertex s, Vertex t) {
    for each v in V - {s} {
        d[v] = INF;
        pi[v] = NIL;
    }
    d[s] = 0;
    pi[s] = NIL;
    A = {s};

    while (A is not empty) {
        v = vertex in A with minimum d[v];
        remove v from A;
        if (v == t) {
            output d[t];
            return;
        }

        for each w adjacent to v {
            if (d[w] == INF) {
                d[w] = d[v] + l(v, w);
                pi[w] = v;
                insert w into A;
            } else if (d[w] > d[v] + l(v, w)) {
                d[w] = d[v] + l(v, w);
                pi[w] = v;
            }
        }
    }
}
```



S に入った頂点から候補距離を更新する

図 24: Dijkstra 法の進み方. 緑の頂点は距離が確定済みであり, そこから出る辺で候補距離を更新する.

10.5 最短路の出力

$\pi(v)$ は最短路において v の直前に来る頂点を表す. したがって, 終点から π を逆向きにたどれば最短路を復元できる.

```
void output_path(Vertex v) {
    Vertex u = v;
    while (pi[u] != NIL) {
        print u;
        u = pi[u];
    }
    print u;
}
```

この手続きは終点から始点へ逆順に出力する. 始点から終点の順に出したい場合は, スタックに積むか再帰で出力する.

10.6 計算量を決める要素

Dijkstra 法の効率は, 主に次の 2 点で決まる.

- グラフ表現: 隣接行列か隣接リストか.
- 候補集合 A の実装: 1 次元配列かヒープか.

実装	計算量の目安
隣接行列 + 1 次元配列	$O(V ^2)$
隣接リスト + 2 分ヒープ	$O(E \log_2 V)$
隣接リスト + フィボナッチヒープ	$O(E + V \log_2 V)$

講義では, 1 次元配列と 2 分ヒープを中心に扱う.

10.7 1 次元配列で候補集合を持つ

配列 I_A を用意し,

$$I_A[v] = \begin{cases} 1 & v \in A, \\ 0 & v \notin A \end{cases}$$

とする. 挿入, 削除, 所属判定は $O(1)$ でできる. しかし, A の中で d が最小の頂点を探すには, すべての頂点を調べる必要があるので $O(|V|)$ かかる. これを各頂点について行うため, 全体では $O(|V|^2)$ になる.

10.8 2 分ヒープで候補集合を持つ

候補集合 A を, key が $d(v)$, 値が頂点 v である最小ヒープとして管理する. 根には候補距離が最小の頂点が入る.

必要なヒープ操作

- Insert: 新しい候補頂点をヒープに挿入する.
- Delete-Min: 根の最小 key を持つ頂点を削除して返す.
- Decrease-Key: 既にヒープに入っている頂点の key を小さくする.

ヒープ内のセルには **key** と **vertex** を持たせる. また, 各頂点 v がヒープのどのアドレスにいるかを $\text{adr}[v]$ に持たせる. 交換時には key だけでなく vertex と adr も同時に更新する.

```
void dijkstra_with_heap(Graph G, Vertex s, Vertex t) {
    for each v in V - {s} {
        d[v] = INF;
        pi[v] = NIL;
        adr[v] = NIL;
    }
    d[s] = 0;
    pi[s] = NIL;
    make heap H empty;
    insert(H, d[s], s);

    while (H is not empty) {
        v = delete_min(H);
        if (v == t) {
            output d[t];
            return;
        }
        for each w adjacent to v {
            if (d[w] == INF) {
                d[w] = d[v] + l(v, w);
                pi[w] = v;
                insert(H, d[w], w);
            } else if (d[w] > d[v] + l(v, w)) {
                d[w] = d[v] + l(v, w);
                pi[w] = v;
                decrease_key(H, adr[w], d[w]);
            }
        }
    }
}
```

10.9 ヒープ操作の注意点

第 4 回のヒープソートでは最大ヒープを使ったが, Dijkstra 法では最小ヒープを使う. したがってヒープ性は

$$\text{key}[\text{parent}(i)] \leq \text{key}[i]$$

である. Upheap-Sort と Downheap-Sort は不等号の向きを反転させて使う.

注意 10.1: adr の更新

Upheap-Sort や Downheap-Sort でセルを交換するとき, `key` と `vertex` だけを交換してはいけない. 各頂点があるヒープアドレス `adr[vertex]` も同時に更新しないと, 後の `Decrease-Key` が間違ったセルを更新する.

10.10 A* アルゴリズム

講義の最後では, Dijkstra 法の発展として A* アルゴリズムが紹介されている. A* は終点までの距離の推定値を使い, 終点に近そうな頂点を優先して探索する. 大規模なネットワークで有用だが, 最適性が保証されるかは推定関数の性質に依存する.

10.11 演習を解くためのチェックリスト

- Dijkstra 法は辺の長さが非負であることを前提にする.
- $d(v)$ は候補距離, $\pi(v)$ は最短路復元用の親である.
- 候補集合 A から d 最小の頂点を取り出す操作が効率を左右する.
- 隣接行列 + 1 次元配列では $O(|V|^2)$ になる.
- 2 分ヒープでは `Insert`, `Delete-Min`, `Decrease-Key` が必要である.
- ヒープ内のセル交換では `adr` も更新する.

11 ユニオン・ファインド森と最小木

11.1 この章で何を理解するか

この章では、集合の分割を効率よく管理するユニオン・ファインド森を学ぶ。その応用として、無向グラフの連結成分分解と、重みつき連結グラフの最小木を求める Kruskal 法を扱う。講義資料のファイル名は第 11 回だが、スライド内タイトルは第 12 回となっている。

この章で固定して読む記号

記号・変数	意味
S	管理したい全体集合.
$p[x]$	ユニオン・ファインド森で x の親を指す配列. 根は自分自身を指す.
$rank[x]$	根付き木の高さの目安. Union-by-rank で使う.
Make-Set	単元集合を作る操作.
Find-Set	要素が属する集合の代表を返す操作.
Union	2 つの集合を統合する操作.
T	Kruskal 法で選ばれた辺集合.
$w(e)$	辺 e の重み.

11.2 連結成分分解の考え方

無向グラフ $G = (V, E)$ の連結成分とは、互いにパスで行き来できる頂点を集めたものである。DFS でも $O(|V| + |E|)$ で求められるが、辺を 1 本ずつ追加する見方でも考えられる。

辺を追加する見方

- 最初は辺がないので、各頂点は単独の連結成分である。
- 辺 (u, v) を追加すると、 u を含む成分と v を含む成分が統合される。
- すでに同じ成分にいる 2 頂点を結ぶ辺を追加しても、成分は変わらない。

```
Connected_Components(G) {
  for each u in V {
    Make_Set(u);
  }
  for each edge (u, v) in E {
    if (Find_Set(u) != Find_Set(v)) {
      Union(u, v);
    }
  }
}
```

このためには、「どの要素がどの部分集合に属するか」を管理し、部分集合どうしを統合するデータ構造が必要である。

11.3 ユニオン・ファインド森

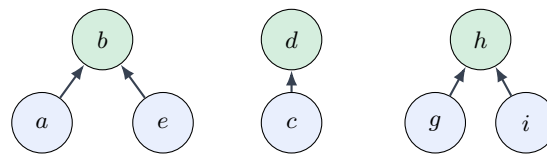
定義 11.1: ユニオン・ファインド森

ユニオン・ファインド森とは、集合の分割を根付き木の集まりとして保持し、Make-Set, Find-Set, Union を効率よく行うデータ構造である。

各部分集合は根付き木で表す。根がその集合の代表であり、根以外の頂点は親を指す。根は自分自身を親として指す。

$$x \text{ が代表} \iff p[x] = x$$

である。



根が集合の代表になる

図 25: ユニオン・ファインド森. 各根付き木が 1 つの部分集合を表し、根が代表である。

11.4 素朴な実装

```
void make_set(Vertex x) {
    p[x] = x;
}

Vertex find_set(Vertex x) {
    Vertex y = x;
    while (y != p[y]) {
        y = p[y];
    }
    return y;
}

void naive_union(Vertex x, Vertex y) {
    Vertex u = find_set(x);
    Vertex v = find_set(y);
    if (u != v) {
        p[u] = v;
    }
}
```

素朴な Union では、木がパス状に伸びることがある。この場合、高さが $O(n)$ になり、Find-Set も $O(n)$ にかかる。

11.5 Union-by-rank

木が高くなりすぎるのを防ぐため、各根に **rank** を持たせる。rank が小さい根を rank が大きい根の下へぶら下げる。rank が等しいときだけ、新しい根の rank を 1 増やす。

```
void make_set(Vertex x) {
    p[x] = x;
    rank[x] = 0;
}

void union_set(Vertex x, Vertex y) {
    Vertex u = find_set(x);
    Vertex v = find_set(y);
    if (u != v) {
        link(u, v);
    }
}

void link(Vertex x, Vertex y) {
    if (rank[x] > rank[y]) {
        p[y] = x;
    } else {
        p[x] = y;
        if (rank[x] == rank[y]) {
            rank[y] = rank[y] + 1;
        }
    }
}
```

rank の性質

- 初期状態ではすべての rank は 0.
- 根でなくなった頂点の rank はそれ以降変化しない.
- Union-by-rank だけを使うと, rank の最大値は $\log_2 n$ 以下になる.
- したがって Find-Set と Union は $O(\log_2 n)$ で行える.

11.6 パス圧縮

Find-Set(x) は x から根までのパスをたどる。このとき、パス上の各頂点の親を根へ直接張り替えると、次回以降の探索が速くなる。これをパス圧縮という。

```
Vertex find_set(Vertex x) {
    Vertex y = x;
    while (y != p[y]) {
        y = p[y];
    }
    Vertex r = y;
```

```

y = x;
while (y != r) {
    Vertex z = p[y];
    p[y] = r;
    y = z;
}
return r;
}

```

注意 11.1: 親を保存してから張り替える

パス圧縮では, $p[y]$ を根に書き換える前に, 元の親を z へ保存する必要がある. 保存せずに書き換えると, 次にたどるべき頂点が分からなくなる.

Union-by-rank とパス圧縮を併用すると, 実用上は各操作をほぼ $O(1)$ と見なせる. 厳密には, n 回の Make-Set と k 回の Union/Find-Set の合計時間は

$$O(n + k\alpha(k, n))$$

である. ここで α は Ackermann 関数の逆関数で, 現実的な入力では 4 以下である.

11.7 木と全域木

定義 11.2: 木

木とは, 連結でサイクルを持たない無向グラフである.

木には次の性質がある. 頂点数を $|V|$ とすると, 辺数は

$$|V| - 1$$

である. また, 連結性, サイクルを持たないこと, 辺数が $|V| - 1$ であることのうち 2 つが成り立てば, 残りも成り立つ.

定義 11.3: 全域木

連結グラフ $G = (V, E)$ の全域木とは, 頂点集合 V はそのままに, 辺集合の一部 $T \subseteq E$ を選んでできる木である.

11.8 最小木問題

定義 11.4: 最小木問題

重みつき連結グラフ $G = (V, E)$ に対し, 辺重みの総和が最小になる全域木を求める問題を最小木問題という.

代表的な解法に Kruskal 法と Prim 法がある. 講義では Kruskal 法を扱う.

11.9 Kruskal 法

Kruskal 法は、辺を重みの小さい順に見て、サイクルを作らないなら採用する貪欲法である。

```
Kruskal(G, w) {  
    sort edges by nondecreasing weight;  
    T = empty set;  
    for each u in V {  
        Make_Set(u);  
    }  
    for each edge e = (u, v) in sorted order {  
        if (Find_Set(u) != Find_Set(v)) {  
            T = T union {e};  
            Union(u, v);  
        }  
        if (|T| == |V| - 1) {  
            break;  
        }  
    }  
    return T;  
}
```

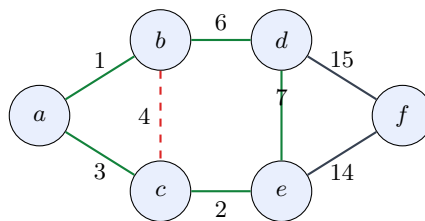


図 26: Kruskal 法の見方. 重みの小さい辺から見て、サイクルを作る辺は捨てる。

11.10 サイクル判定とユニオン・ファインド

現在選んだ辺集合を T とする. $T \cup \{e\}$ がサイクルを持つかどうかは、辺 $e = (u, v)$ の両端が、 T で作られるグラフの同じ連結成分に入っているかで判定できる。

$$u, v \text{ が同じ成分} \iff T \cup \{(u, v)\} \text{ はサイクルを持つ}$$

したがって、Kruskal 法では連結成分をユニオン・ファインド森で管理すればよい。

11.11 Kruskal 法の正当性

Kruskal 法が最小木を返すことは、交換論法で示せる. Kruskal 法で得た木を T , 最小木を T^* とし、 T^* は T と共通する辺数が最大の最小木だとする. T には入っていて T^* には入っていない最初の辺を e_k とする。

T^* に e_k を加えるとサイクルができる。そのサイクルには T に入っていない辺 e_j がある。 e_k は Kruskal 法で先に選ばれたので

$$w(e_k) \leq w(e_j)$$

である。そこで $T^* + e_k - e_j$ を作ると、重みは増えず、しかも T と共通する辺が 1 本増える。これは T^* の選び方に矛盾する。よって T は最小木である。

11.12 Kruskal 法の計算量

辺数を $m = |E|$ 、頂点数を $n = |V|$ とする。辺のソートに

$$O(m \log_2 m) = O(m \log_2 n)$$

かかる。ユニオン・ファインド森の操作はほぼ線形時間で済むため、全体の支配項はソートである。したがって Kruskal 法の計算量は

$$O(m \log_2 n)$$

である。

11.13 演習を解くためのチェックリスト

- Find-Set(x) は x が属する集合の代表を返す。
- Union-by-rank では rank が低い根を高い根へぶら下げる。
- パス圧縮では、根までのパス上の頂点を直接根へつなぎ直す。
- 木は連結かつサイクルなしで、辺数は頂点数より 1 少ない。
- Kruskal 法では、重みの小さい辺から見てサイクルを作らない辺だけ採用する。
- サイクル判定は、辺の両端が同じ連結成分にあるかで行う。
- Kruskal 法の計算量は辺ソートが支配し $O(m \log_2 n)$ である。

12 動的計画法

12.1 この章で何を理解するか

この章では、最適化問題を小問題に分け、同じ小問題を何度も解かないように表へ保存する動的計画法を学ぶ。題材は全頂点对間最短路問題、連鎖行列積問題、弾性マッチングである。共通する考え方は、最適性の原理と重なり合う小問題である。

この章で固定して読む記号

記号・変数	意味
$d^k[i, j]$	Floyd-Warshall 法で、経由点を $\{1, \dots, k\}$ に制限した i から j への最短距離。
$l(i, j)$	辺 (i, j) の長さ。辺がなければ ∞ 。
A_i	連鎖行列積の i 番目の行列。サイズは $p_{i-1} \times p_i$ 。
$m[i, j]$	$A_i A_{i+1} \cdots A_j$ を計算する最小コスト。
$s[i, j]$	最適な分割位置。
$c(i, j)$	弾性マッチングで a_i と b_j を対応させるコスト。
$D[i, j]$	弾性マッチングの部分問題の最小コスト。

12.2 最適性の原理

定義 12.1: 最適性の原理

最適性の原理とは、最適解の一部分だけに注目しても、その部分が対応する小問題の最適解になっているという性質である。

この性質が成り立ち、さらに小問題が重なり合う場合、動的計画法が有効である。大きな問題を直接再帰で解くと同じ小問題を何度も解いてしまうが、表に保存すれば一度だけ計算すればよい。

動的計画法の設計手順

- 最適解の構造を特徴づける。
- 最適値の再帰式を作る。
- 小さい小問題から表を埋める。
- 必要なら表から最適解そのものを復元する。

12.3 全頂点对間最短路問題

定義 12.2: 全頂点对間最短路問題

重みつきグラフ $G = (V, E)$ に対し、すべての頂点对 (i, j) について、 i から j への最短距離を求める問題を全頂点对間最短路問題という。

Floyd-Warshall 法は、経由点として使ってよい頂点集合を少しずつ増やしていく。頂点集合を $V = \{1, 2, \dots, n\}$ とする。

12.4 Floyd-Warshall 法

$d^k[i, j]$ を, 経由点として $\{1, 2, \dots, k\}$ だけを使ってよいときの, i から j への最短距離とする. 最短路が頂点 k を通らないなら $d^{k-1}[i, j]$ でよい. 頂点 k を通るなら, i から k と k から j に分けられる.

したがって,

$$d^k[i, j] = \min\{d^{k-1}[i, j], d^{k-1}[i, k] + d^{k-1}[k, j]\}$$

である.

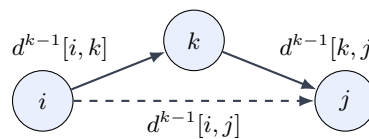
```
void floyd_warshall(int n, real l[][]) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            d[0][i][j] = l[i][j];
        }
        d[0][i][i] = 0;
    }

    for (int k = 1; k <= n; k++) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                d[k][i][j] = min(d[k-1][i][j],
                                d[k-1][i][k] + d[k-1][k][j]);
            }
        }
    }
}
```

3 重ループなので計算量は

$$O(n^3)$$

である. 有向グラフでもほぼ同じ形で使えるが, 一般には $d^k[i, j] \neq d^k[j, i]$ なので全ての組 (i, j) を見る.



頂点 k を通るか通らないかを比較する

図 27: Floyd-Warshall 法の再帰式. 新しく許す頂点 k を通る経路と通らない経路を比較する.

12.5 最短路そのものの復元

距離だけでなく経路も求めるには, 直前頂点を表す表 $\pi^k[i, j]$ を持つ. 更新が

$$d^k[i, j] = d^{k-1}[i, k] + d^{k-1}[k, j]$$

で起きたなら, i から j への最短路の最後の直前頂点は, k から j への最短路の直前頂点と同じである. Dijkstra 法の π と同じように, 終点側からたどれば経路を復元できる.

12.6 行列積の計算順

行列 A が $p \times q$, 行列 B が $q \times r$ のとき, 積 AB は $p \times r$ 行列であり, 計算手間はおよそ

$$pqr$$

に比例する.

3 つ以上の行列積では, カッコの付け方で手間が大きく変わる. たとえば $A : p \times q, B : q \times r, C : r \times s$ なら,

$$(AB)C : pqr + prs, \quad A(BC) : qrs + pqs$$

である.

12.7 連鎖行列積問題

定義 12.3: 連鎖行列積問題

行列 $A_1, A_2, \dots, A_n$ が与えられ, A_i のサイズが $p_{i-1} \times p_i$ であるとき, 積 $A_1 A_2 \cdots A_n$ を計算する最小手間のカッコづけを求める問題を連鎖行列積問題という.

$m[i, j]$ を $A_i A_{i+1} \cdots A_j$ を計算する最小手間とする. $i = j$ なら行列 1 個なので

$$m[i, i] = 0$$

である. $i < j$ のとき, 最後の積を k で分けると,

$$(A_i \cdots A_k)(A_{k+1} \cdots A_j)$$

となる. 左側のサイズは $p_{i-1} \times p_k$, 右側のサイズは $p_k \times p_j$ なので, 最後の積のコストは

$$p_{i-1} p_k p_j$$

である.

したがって,

$$m[i, j] = \min_{k=i, \dots, j-1} \{m[i, k] + m[k+1, j] + p_{i-1} p_k p_j\}$$

となる.

12.8 Matrix-Chain-Order

```
void matrix_chain_order(int n, int p[]) {
    for (int i = 1; i <= n; i++) {
        m[i][i] = 0;
    }

    for (int len = 2; len <= n; len++) {
        for (int i = 1; i <= n - len + 1; i++) {
            int j = i + len - 1;
            m[i][j] = INF;
            for (int k = i; k <= j - 1; k++) {
                int q = m[i][k] + m[k+1][j] + p[i-1] * p[k] * p[j];
                if (q < m[i][j]) {
                    m[i][j] = q;
                    s[i][j] = k;
                }
            }
        }
    }
}
```

3 重ループなので計算量は

$$O(n^3)$$

である。 $s[i, j]$ には最適な分割位置を保存する。これを再帰的に読めば最適なカッコづけを復元できる。

全列挙が難しい理由

n 個の行列積のカッコづけ数は Catalan 数で増え、おおよそ指数的に大きくなる。そのため全列挙は現実的でない。動的計画法では、 $O(n^2)$ 個程度の小問題を表に保存し、 $O(n^3)$ で最適値を求める。

12.9 動的計画法と分割統治法の違い

分割統治法では、子問題どうしが基本的に重ならない。たとえば Merge-Sort の左半分と右半分は別々の配列区間である。

一方、連鎖行列積を素朴な再帰で解くと、同じ $m[i, j]$ が何度も必要になる。このように小問題が重なるとき、結果を表に保存する動的計画法が有効である。

12.10 弾性マッチング

2 つの列

$$A = (a_1, a_2, \dots, a_m), \quad B = (b_1, b_2, \dots, b_n)$$

を対応づける問題を考える。各ペア (a_i, b_j) にはコスト $c(i, j)$ がある。

定義 12.4: 弾性マッチング

弾性マッチングとは、2つの列の要素を、順序を乱さず、各要素が少なくとも1つのペアに含まれるように対応づける方法である。

対応を格子上的パスとして見ると、 $(1, 1)$ から (m, n) へ、右・上・斜めに進む後戻りなしのパスになる。

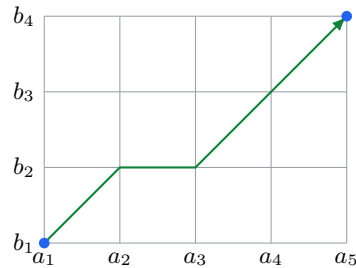


図 28: 弾性マッチングと格子パス. 順序を乱さない対応は、後戻りしないパスとして表せる。

12.11 弾性マッチングの再帰式

$D[i, j]$ を、 $a_1, \dots, a_i$ と $b_1, \dots, b_j$ の弾性マッチングの最小コストとする。境界条件は

$$D[1, 1] = c(1, 1),$$

$$D[i, 1] = D[i-1, 1] + c(i, 1), \quad D[1, j] = D[1, j-1] + c(1, j)$$

である。 $i, j > 1$ では、最後に (a_i, b_j) を対応させる直前は、

$$(i-1, j-1), \quad (i-1, j), \quad (i, j-1)$$

のいずれかである。したがって、

$$D[i, j] = \min\{D[i-1, j-1], D[i-1, j], D[i, j-1]\} + c(i, j)$$

である。

```
void elastic_matching(int m, int n, real c[][]) {
    D[1][1] = c[1][1];
    for (int i = 2; i <= m; i++) {
        D[i][1] = D[i-1][1] + c[i][1];
    }
    for (int j = 2; j <= n; j++) {
        D[1][j] = D[1][j-1] + c[1][j];
    }
    for (int i = 2; i <= m; i++) {
        for (int j = 2; j <= n; j++) {
            D[i][j] = min(D[i-1][j-1], D[i-1][j], D[i][j-1]) + c[i][j];
        }
    }
}
```

小問題数は mn 個なので、計算量は

$$O(mn)$$

である。

12.12 メモ化

定義 12.5: メモ化

メモ化とは、再帰的なトップダウン計算で、一度計算した小問題の値を表に保存し、次に同じ小問題が呼ばれたときは表の値を返す方法である。

動的計画法は小さい問題から表を埋めるボトムアップ型であることが多い。メモ化は自然な再帰構造を保ちつつ、同じ小問題の重複計算を避ける。連鎖行列積のメモ化版も $O(n^3)$ で動くが、定数倍はボトムアップ版より大きいことが多い。

12.13 演習を解くためのチェックリスト

- Floyd-Warshall 法では、頂点 k を経由するかどうかで再帰式を作る。
- $d^k[i, j] = \min\{d^{k-1}[i, j], d^{k-1}[i, k] + d^{k-1}[k, j]\}$ を説明できるようにする。
- 連鎖行列積では、最後の分割位置 k を全て試す。
- $m[i, j] = \min_k\{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\}$ を使えるようにする。
- $s[i, j]$ から最適なカッコづけを復元できることを確認する。
- 弾性マッチングは格子上の後戻りなしパスとして見られる。
- 動的計画法が効く条件は、最適性の原理と重なり合う小問題である。